How to Use the IEEEtran LaTeX Class

Michael Shell, *Member, IEEE*

(Invited Paper)

Abstract—This article describes how to use the IEEEtran class with LaTeX to produce high quality typeset papers that are suitable for submission to the Institute of Electrical and Electronics Engineers (IEEE). IEEEtran can produce conference, journal and technical note (correspondence) papers with a suitable choice of class options. This document was produced using IEEEtran in Computer Society (compsoc) journal mode.

Index Terms—Class, IEEEtran, LaTeX, paper, style, template, typesetting.

1 Introduction

With a recent IEEEtran class file, a computer running LaTeX, and a basic understanding of the LaTeX language, an author can produce professional quality typeset research papers very quickly, inexpensively, and with minimal effort. The purpose of this article is to serve as a user guide of IEEEtran LaTeX class and to document its unique features and behavior.

This document applies to version 1.8b and later of IEEEtran. Prior versions do not have all of the features described here. IEEEtran will display the version number on the user's console when a document using it is being compiled. The latest version of IEEEtran and its support files can be obtained from IEEE's web site [1], or CTAN [2]. This latter site may have some additional material, such as beta test versions and files related to non-IEEE uses of IEEEtran. See the IEEEtran homepage [3] for frequently asked questions and recent news about IEEEtran.

Complimentary to this document are the files[1] `bare_conf_compsoc.tex` and `bare_jrnl_compsoc.tex`, which are "bare bones" example (template) files of an IEEE Computer Society conference and journal paper, respectively. Authors can quickly obtain a functional document by using these files as starters for their own work. A more advanced example featuring the use of optional packages along with more complex usage techniques, can be found in `bare_adv.tex`.

It is assumed that the reader has at least a basic working knowledge of LaTeX. Those so lacking are strongly encouraged to read some of the excellent literature on the subject [4]–[6]. In particular, Tobias Oetiker's *The Not So Short Introduction to LaTeX 2ε* [5], which provides a general overview of working with LaTeX, and Stefan M. Moser's *How to Typeset Equations in LaTeX* [6], which focuses on the formatting of IEEE-style equations using IEEEtran's IEEEeqnarray commands, are both available for free online.

General support for LaTeX related questions can be obtained in the internet newsgroup comp.text.tex. There is also a searchable list of frequently asked questions about LaTeX [7].

Please note that the appendices sections contain information on installing the IEEEtran class file as well as tips on how to avoid commonly made mistakes.

2 Class Options

There are a number of class options that can be used to control the overall mode and behavior of IEEEtran. These are specified in the traditional LaTeX way. For example,

```
\documentclass[9pt,technote]{IEEEtran}
```

is used with correspondence/brief/technote papers. The various categories of options will now be discussed. For each category, the default option is shown in italics. The user must specify an option from each category in which the default is not the one desired. The various categories are totally orthogonal to each other—changes in one will not affect the defaults in the others.

2.1 9pt, *10pt*, 11pt, 12pt

There are four possible values for the normal text size. 10pt is used by the vast majority of papers. Notable exceptions are technote papers, which use 9pt text and the initial submissions to some conferences that use 11pt.

Be aware that IEEE Computer Society publications use "PostScript" (i.e., "big point", bp) point sizes (i.e., 72bp = 1in) rather than the traditional typesetters' point (i.e., 72.27pt = 1in). Also, "10pt" IEEE Computer Society journal papers actually use a slightly smaller, 9.5bp, font size (probably to compensate for the slightly wider nature of the Palatino font). IEEEtran will automatically tweak the selected font size as needed depending on the mode.

• *See http://www.michaelshell.org/ for current contact information.*

Manuscript created February 25, 2002; revised August 26, 2015. This work was supported by the IEEE. *This work is distributed under the LaTeX Project Public License (LPPL) (http://www.latex-project.org/) version 1.3. A copy of the LPPL, version 1.3, is included in the base LaTeX documentation of all distributions of LaTeX released 2003/12/01 or later. The opinions expressed here are entirely that of the author. No warranty is expressed or implied. User assumes all risk.*

1. Note that it is the convention of this document not to hyphenate command or file names and to display them in `typewriter font`. Within such constructs, spaces are not implied at a line break and will be explicitly carried into the beginning of the next line. This behavior is not a feature of IEEEtran, but is used here to illustrate computer commands verbatim.

0000–0000/00$00.00 © 2015 Michael Shell

2.2 draft, draftcls, draftclsnofoot, *final*

IEEEtran provides for three draft modes as well as the normal final mode. The draft modes provide a larger (double) line spacing to allow for editing comments as well as one inch margins on all four sides of the paper. The standard draft option puts *every* package used in the document into draft mode. With most graphics packages, this has the effect of disabling the rendering of figures. If this is not desired, one can use the draftcls option instead to yield a draft mode that will be confined within the IEEEtran class so that figures will be included as normal. draftclsnofoot is like draftcls, but does not display the word "DRAFT" along with the date at the foot of each page. Both draft and draftclsnofoot modes imply draftcls (which is a subset of the other two). When using one of the draft modes, most users will also want to select the onecolumn option.

2.3 conference, *journal*, technote, peerreview, peerreviewca

IEEEtran offers five major modes to encompass conference, journal, correspondence (brief/technote) and peer review papers. Journal and technote modes will produce papers very similar to those that appear in many IEEE TRANSACTIONS journals. When using technote, most users should also select the 9pt option. The peerreview mode is much like the journal mode, but produces a single-column cover page (with the title, author names and abstract) to facilitate anonymous peer review. The title is repeated (without the author names or abstract) on the first page after the cover page.[2] Papers using the peer review options require an `\IEEEpeerreviewmaketitle` command (in addition to and after the traditional `\maketitle`) to be executed at the place the cover page is to end—usually just after the abstract. This command will be silently ignored with the non-peerreview modes. See the bare template files for an example of the placement of this command. The peerreviewca mode is like peerreview, but allows the author name information to be entered and formatted as is done in conference mode (see Section 4.2.2 for details) so that author affiliation and contact information is more visible to the editors.

2.3.1 Conference Mode Details

Conference mode makes a number of significant changes to the way IEEEtran behaves:

- The margins are increased as the height of the text is reduced to about 9.25in. In particular, the bottom margin will become larger than that of the top as the IEEE wants extra clearance at the bottom. The text height will not be exactly 9.25in, but will vary slightly with the normal font size to ensure an integer number of lines in a column.
- Headings and page numbers are not displayed in the headers or footers. This, coupled with symmetric horizontal margins, means that there will not be a noticeable difference between the one and two sided options.
- The `\author` text is placed within a tabular environment to allow for multicolumn formatting of author names and affiliations. Several commands are enabled to facilitate this formatting (see Section 4.2.2 for details).
- The spacing after the authors' names is reduced. So is the spacing around the section names.
- The special paper notice (if used) will appear *between* the author names and the title (not after as with journals).
- The figure captions are centered.
- The following commands are intentionally disabled: `\thanks`, `\IEEEPARstart`, `\IEEEbiography`, `\IEEEbiographynophoto`, `\IEEEpubid`, `\IEEEpubidadjcol`, `\IEEEmembership`, and `\IEEEaftertitletext`. If needed, they can be reenabled by issuing the command: `\IEEEoverridecommandlockouts`.
- Various reminder (related to camera ready work) and warning notices are enabled.

When using conference mode, most users will also want to equalize the columns on the last page (see Section 14).

2.4 comsoc, compsoc, transmag

These mutually exclusive options invoke special modes by which IEEEtran produces the format of the publications of the IEEE Communications Society, IEEE Computer Society and IEEE TRANSACTIONS ON MAGNETICS, respectively. Neither of these are enabled by default.

2.4.1 Comsoc Mode

Comsoc mode only affects the math font so that it will more closely match the Times Roman main text. Either Michael Sharpe's freely available newtxmath package [8] (version 1.451, July 28, 2015 or later is recommended) or the commercial MathTime [9] math fonts (as mtpro2.sty, mt11p.sty or mathtime.sty) are acceptable. Under comsoc mode, if one of these packages has not been loaded by the user at the start of the document, IEEEtran will attempt to enforce their use based on what is available on the system.

The recommended loading procedure and order for newtxmath is:

```
\usepackage[T1]{fontenc} % optional
\usepackage{amsmath}
\usepackage[cmintegrals]{newtxmath}
\usepackage{bm} % optional
```

where the cmintegrals option, which IEEEtran sets as a default upon loading newtxmath, is needed to obtain the specific style of integral symbol used by the IEEE Communications Society. The optional bm package [10] provides for selective bold math. Be aware that the AMS Math amssymb.sty package [11] is not needed and should not be loaded as that functionality is built into and provided by newtxmath as well as MathTime. Also, do not load the newtxtext.sty package as doing so would alter the main text font.

2.4.1.1 Comsoc Conference Mode: Comsoc conference papers are, at present, done the same way as traditional conference papers (`bare_conf.tex`) and so no additional example file is required. Unless specifically instructed otherwise by the conference that is being submitting to, do not invoke the comsoc option with conference papers.

2. A blank page may be inserted after the cover page when using the twoside (duplex printing) option so that the beginning of the paper does not appear on the back side of the cover page.

2.4.2 Compsoc Mode

Notable compsoc mode format features include:

- the default text font is changed from Times Roman to Palatino/Palladio (non-conference compsoc modes only);
- revised margins;
- Arabic section numbering;
- enabling of the `\IEEEcompsocitemizethanks` and `\IEEEcompsocthanksitem` commands to provide for the `\thanks` (first footnote) itemized list used for author affiliations;
- enabling of the `\IEEEtitleabstractindextext` command to provide for single column abstract and index terms (see Section 5);
- various other styling changes (most of which are only applicable under the non-conference compsoc modes) such as the use of: a sans serif (Helvetica) font for titles, headings, etc.; a ruled line above the first footnote area; left aligned reference labels; etc.

2.4.2.1 Compsoc Conference Mode: IEEEtran follows the guidelines for IEEE Computer Society conference papers. Perhaps surprisingly, this format nullifies many of the unique features of compsoc journals and is not so much different from traditional conference mode. However, Arabic section numbering is retained. It should be mentioned that Scott Pakin's IEEEconf LaTeX class [12] also produces this format. Be aware that many IEEE Computer Society conferences use the traditional conference format and compsoc mode should not be used with them.

2.4.3 Transmag Mode

For the transmag mode:

- The text within `\author` should be entered as the long form under conference mode;
- enabling of the `\IEEEtitleabstractindextext` command to provide for single column abstract and index terms (see Section 5);
- `\IEEEauthorrefmark` will produce arabic author affiliation symbols;
- subsection and subsubsection headings and/or their spacings are slightly different;
- a smaller, bold font than normal is used for the title.

The transmag mode (as well as the standard journal mode) is also acceptable for submission to *IEEE Magnetics Letters*. Authors who wish to have their figures and tables appear at the end of the paper can use the endfloat.sty [13] package to achieve this.

2.5 *letterpaper*, a4paper, cspaper

IEEEtran fully supports both the US letter (8.5in × 11in) and A4 (210mm × 297mm) paper sizes. Since the IEEE primarily uses US letter, authors should usually select the letterpaper option before submitting their work to the IEEE—unless told otherwise (typically by conferences held outside the United States). Changing the paper size in the standard journal and conference modes will *not* alter the typesetting of the document—only the margins will be affected. In particular, documents using the a4paper option will have reduced side margins (A4 is narrower than US letter) and a longer bottom margin (A4 is longer than US letter). For both cases, the top margins will be the same and the text will be horizontally centered.

For the compsoc conference and draft modes, it is the margins that will remain constant, and thus the text area size will vary, with changes in the paper size.

The cspaper option is the special "trim" paper size (7.875in × 10.75in) used in the actual publication of IEEE Computer Society journals. Under compsoc journal mode, this option does not alter the typesetting of the document. Authors should invoke this option only if requested to do so by the editors of the specific journal they are submitting to.

Note that authors should ensure that all post-processing (PS, PDF, etc.) uses the same paper specification as the .tex document. Problems here are by far the number one reason for incorrect margins. See Appendix B for more details.

For the special cspaper size, be aware that although IEEEtran will automatically configure the correct paper dimensions for pdfLaTeX's PDF mode (which it does for all paper sizes), dvips (the application used for DVI to PS conversion) systems will not recognize the special "ieeecs" paper unless there is such an entry in dvips' `config.ps` configuration file:

```
% Special paper size for the IEEE Computer Society J
ournals
@ ieeecs 7.875in 10.75in
@+ ! %%DocumentPaperSizes: ieeecs
@+ %%BeginPaperSize: ieeecs
@+ /setpagedevice where
@+  { pop << /PageSize [567 774] >> setpagedevice }
@+ if
@+ %%EndPaperSize
```

Most modern PS to PDF conversion software will correctly handle such custom paper sizes if a different specific paper size is not explicitly requested for the conversion process.

2.6 *oneside*, twoside

These options control whether the layout follows that of single sided or two sided (duplex) printing. Because the side margins are normally centered, the main notable difference is in the format of the running headings.

2.7 onecolumn, *twocolumn*

These options allow the user to select between one and two column text formatting. Since the IEEE always uses two column text, the onecolumn option is of interest only with draft papers.

2.8 romanappendices

IEEEtran defaults to numbering appendices alphabetically (e.g., A, B, etc.). Invoke this option to get Roman numbering.

2.9 captionsoff

Invoking this option will inhibit the display of captions within figures and tables. This is done in a manner that preserves the operation of `\label` within `\caption`. This option is intended for journals, such as IEEE TRANSACTIONS ON POWER ELECTRONICS (TPE), that require figures

and tables to placed, captionless, on pages of their own at the end of the document. Such figure placement can be achieved with the help of the endfloat.sty package [13]:

```
\usepackage[nomarkers]{endfloat}
```

Note that the TPE has other unusual formatting requirements that also require the draftclassnofoot and onecolumn options as well as the insertion of page breaks (`\newpage`) just prior to the first section as well as the bibliography. Such commands can be enabled conditionally via the `\ifCLASSOPTIONcaptionsoff` conditional (Section 3.1).

2.10 nofonttune

IEEEtran normally alters the default interword spacing to be like that used in IEEE publications. The result is text that requires less hyphenation and generally looks more pleasant, especially for two column text. The nofonttune option will disable the adjustment of these font parameters. This option should be of interest only to those who are using fonts specifically designed or modified for use with two column work.

3 The CLASSINPUT, CLASSOPTION and CLASSINFO Controls

IEEEtran offers three catagories of special commands that allow information to be passed between the class file and the user's document:

- CLASSINPUTs are inputs that provide a way to customize the operation of IEEEtran by overriding some of the default settings (at the time IEEEtran is loaded);
- CLASSOPTIONs which are outputs that allow for conditional compilation based on which IEEEtran class options have been selected;
- CLASSINFOs which are outputs that allow the user a way to access additional information about the IEEEtran runtime environment.

3.1 CLASSINPUTs

The available CLASSINPUTs include: `\CLASSINPUTbaselinestretch` which sets the line spacing of the document; `\CLASSINPUTinnersidemargin` which sets the margin at the inner (binding) edge; `\CLASSINPUToutersidemargin` which sets the margin at the outer edge; `\CLASSINPUTtoptextmargin` which sets the top margin; `\CLASSINPUTbottomtextmargin` which sets the bottom margin. Of course, such parameters can be set via the traditional LaTeX interface (`\oddsidemargin`, `\topmargin`, etc.). However, the advantage of of using the CLASSINPUT approach is that it allows IEEEtran to adjust other internal parameters and perform any additional calculations as needed. For example, setting the side margins in LaTeX requires a careful setting of `\oddsidemargin`, `\evensidemargin` and `\textwidth` taking into consideration the paper size and whether or not duplex (two-sided) printing is being used.

To invoke a CLASSINPUT, just define the relavant CLASSINPUT as desired *prior* to the loading of IEEEtran. For example,

```
\newcommand{\CLASSINPUTinnersidemargin}{17mm}
\documentclass{IEEEtran}
```

will yield a document that has 17mm side margins—if only one of the innerside/outerside (or toptext/bottomtext) margin pair is specified, IEEEtran will assume the user wants symmetric side (or top/bottom) margins and will set both values of the relavant pair to the (single) user specified value.

IEEEtran uses the fixed values of 12pt and 0.25in for `\headheight` and `\headsep`, respectively. The position of the header can be altered after IEEEtran is loaded, without changing the margins as long as the sum of `\topmargin`, `\headheight` and `\headsep` is preserved. For example, the header can be shifted upwards 0.2in using:

```
\addtolength{\headsep}{0.2in}
\addtolength{\topmargin}{-0.2in}
```

Likewise, `\footskip`, which has a default value of 0.4in, can easily be changed to alter the position of the footer within the bottom margin.

When using `\CLASSINPUTbaselinestretch`, IEEEtran will automatically "digitize" `\textheight` so that an integer number of lines will fit on a page (as is done in the draft modes). Digitization is not done when the top or bottom margins are set via CLASSINPUTs. Users are cautioned that using CLASSINPUT controls can result in documents that are not compliant with the IEEE's standards. The intended applications include: (1) conferences or societies that have unusual formatting requirements; (2) producing copies with nonstandard margins such as when binding for personal use; and (3) non-IEEE related work.

3.2 CLASSOPTIONs

CLASSOPTIONs are primarily TeX `\if` conditionals that are automatically set based on which IEEEtran options are being used. Thus, for example, a construct such as

```
\ifCLASSOPTIONconference
  \typeout{in conference mode}
\else
  \typeout{not in conference mode}
\fi
```

can be used to provide for conditional code execution. Please note that, as mentioned in Section 2.2, the draft and draftclsnofoot options imply draftcls. So, most users will want to test `\ifCLASSOPTIONdraftcls` for detecting the draft modes.

For the document's point size options, `\CLASSOPTIONpt` is defined as a macro that expands to the numerical part of the selected point value (e.g., 9, 10, 11 or 12). For the paper size options, `\CLASSOPTIONpaper` will be a macro that contains the paper specification (e.g., letter, a4). To use these as conditionals will require a string macro comparison:

```
\newcommand{\myninestring}{9}
\ifx\CLASSOPTIONpt\myninestring
  \typeout{document is 9pt}
\fi
```

Users should treat the CLASSOPTIONs as being "read-only" and not attempt to manually alter their values because IEEEtran uses them internally as flags to determine which options have been selected—changing these flags will likely result in improper formatting.

3.3 CLASSINFOs

The available CLASSINFOs include the `\ifCLASSINFOpdf` conditional which works much like Heiko Oberdiek's ifpdf.sty package [14] to indicate if PDF output (from pdfLaTeX) is in effect:

```
\ifCLASSINFOpdf
  \typeout{PDF mode}
\fi
```

IEEEtran.cls also provides the lengths `\CLASSINFOnormalsizebaselineskip`, which is the `\baselineskip` of the normalsize font, and `\CLASSINFOnormalsizeunitybaselineskip`, which is the `\baselineskip` of the normalsize font under unity `\baselinestetch`.

Finally, there are the string macros (these are not conditionals or lengths) `\CLASSINFOpaperwidth` and `\CLASSINFOpaperheight` which contain the paper dimensions in their native specifications including units (e.g., 8.5in, 22mm, etc.). As with CLASSOPTIONs, users should not attempt to alter the CLASSINFOs.

4 The Title Page

The parts of the document unique to the title area are created using the standard LaTeX command `\maketitle`. Before this command is called, the author must declared all of the text objects which are to appear in the title area.

4.1 Paper Title

The paper title is declared like:

```
\title{A Heuristic Coconut-based Algorithm}
```

in the standard LaTeX manner. Titles are generally capitalized except for words such as a, an, and, as, at, but, by, for, in, nor, of, on, or, the, to and up, which are usually not capitalized unless they are the first or last word of the title. Line breaks (`\\`) may be used to equalize the length of the title lines. Do not use math or other special symbols in the title.

4.2 Author Names

The name and associated information is declared with the `\author` command. `\author` behaves slightly differently depending on the document mode.

4.2.1 Names in Journal/Technote Mode

A typical `\author` command for a journal or technote paper looks something like this:

```
\author{Michael~Shell,~\IEEEmembership{Member,~IEEE,
} John~Doe,~\IEEEmembership{Fellow,~OSA,} and~Jane~D
oe,~\IEEEmembership{Life~Fellow,~IEEE}%
\thanks{Manuscript received January 20, 2002; revise
d August 26, 2015. This work was supported by the IE
EE.}%
\thanks{M. Shell was with the Georgia Institute of T
echnology.}}
```

The `\IEEEmembership` command is used to produce the italic font that indicates the authors' IEEE membership status. The `\thanks` command produces the "first footnotes." Because the LaTeX `\thanks` was not designed to contain multiple paragraphs[3], authors will have to use a separate `\thanks` for each paragraph. However, if needed, regular line breaks (`\\`) can be used within `\thanks`. In order to get proper line breaks and spacing, it is important to correctly use and control the spaces within `\author`. Use nonbreaking spaces (`~`) to ensure that name/membership pairs remain together. A minor, but easy, mistake to make is to forget to prevent unwanted spaces from getting between commands which use delimited (`{}`) arguments. Note the two `%` which serve to prevent the code line break on lines ending in a `}` from becoming an unwanted space. Such a space would not be ignored as an end-of-line space because, technically, the last `\thanks` is the final command on the line. "Phantom" spaces like these would append to the end of the last author's name, causing the otherwise centered name line to shift very slightly to the left.

3. Although IEEEtran.cls does support it, the standard classes do not.

4.2.2 Names in Conference Mode

The author name area is more complex when in conference mode because it also contains the authors' affiliations. For this reason, when in conference mode, the contents of `\author{}` are placed into a modified tabular environment. The commands `\IEEEauthorblockN{}` and `\IEEEauthorblockA{}` are also provided so that it is easy to correctly format the author names and affiliations, respectively. For papers with three or less affiliations, a multicolumn format is preferred:

```
\author{\IEEEauthorblockN{Michael Shell}
\IEEEauthorblockA{School of Electrical and\\
Computer Engineering\\
Georgia Institute of Technology\\
Atlanta, Georgia 30332--0250\\
Email: mshell@ece.gatech.edu}
\and
\IEEEauthorblockN{Homer Simpson}
\IEEEauthorblockA{Twentieth Century Fox\\
Springfield, USA\\
Email: homer@thesimpsons.com}
\and
\IEEEauthorblockN{James Kirk\\
and Montgomery Scott}
\IEEEauthorblockA{Starfleet Academy\\
San Francisco, California 96678-2391\\
Telephone: (800) 555--1212\\
Fax: (888) 555--1212}}
```

Use `\and` to separate the affiliation columns. The columns will automatically be centered with respect to each other and the side margins.

If there are more than three authors and/or the text is too wide to fit across the page, use an alternate long format:

```
\author{\IEEEauthorblockN{Michael Shell\IEEEauthorre
fmark{1}, Homer Simpson\IEEEauthorrefmark{2}, James K
irk\IEEEauthorrefmark{3}, Montgomery Scott\IEEEautho
rrefmark{3} and Eldon Tyrell\IEEEauthorrefmark{4}}
\IEEEauthorblockA{\IEEEauthorrefmark{1}School of Ele
ctrical and Computer Engineering\\
Georgia Institute of Technology, Atlanta, Georgia 30
332--0250\\
Email: mshell@ece.gatech.edu}
\IEEEauthorblockA{\IEEEauthorrefmark{2}Twentieth Cen
tury Fox, Springfield, USA\\
Email: homer@thesimpsons.com}
\IEEEauthorblockA{\IEEEauthorrefmark{3}Starfleet Aca
demy, San Francisco, California 96678-2391\\
Telephone: (800) 555--1212, Fax: (888) 555--1212}
\IEEEauthorblockA{\IEEEauthorrefmark{4}Tyrell Inc.,
```

```
 123 Replicant Street, Los Angeles, California 90210
--4321}}
```

The `\IEEEauthorrefmark{}` command will generate a footnote symbol corresponding to the number in its argument. Use this to link the author names to their respective affiliations. It is not necessary prevent spaces from being between the `\IEEEauthorblock`'s because each block starts a new group of lines and LaTeX will ignore spaces at the very end and beginning of lines.

4.2.3 Names in Compsoc Journal Mode

One unique feature of IEEE Computer Society journals is that author affiliations are formatted in an itemized list within the first (`\thanks`) footnote. In compsoc mode, IEEEtran provides a special form of `\thanks`, `\IEEEcompsocitemizethanks`, to obtain this effect:

```
\author{Michael~Shell,~\IEEEmembership{Member,~IEEE,
} John~Doe,~\IEEEmembership{Fellow,~OSA,} and~Jane~D
oe,~\IEEEmembership{Life~Fellow,~IEEE}%
\IEEEcompsocitemizethanks{\IEEEcompsocthanksitem M.
 Shell is with the Georgia Institute of Technology.
\IEEEcompsocthanksitem J. Doe and J. Doe are with An
onymous University.}%
\thanks{Manuscript received January 20, 2002; revise
d August 26, 2015.}}
```

Within `\IEEEcompsocitemizethanks`, `\IEEEcompsocthanksitem` works like `\item` to provide a bulleted affiliation group. To facilitate dual compilation, in non-compsoc mode, IEEEtran treats `\IEEEcompsocitemizethanks` as `\thanks` and sets `\IEEEcompsocthanksitem` to generate a line break with indentation. However, this is not entirely satisfactory as IEEE Computer Society journals place the author affiliations before the "manuscript received" line while traditional IEEE journals use the reverse order. If correct dual compilation is needed, the CLASSOPTION conditionals can be employed to swap the order as needed.

4.2.4 Names in Compsoc Conference Mode

Names in compsoc conference mode are done in the same way as traditional conference mode.

4.2.5 Names in Transmag Journal Mode

IEEE TRANSACTIONS ON MAGNETICS papers typically use the conference long format for author names, but try to keep each name and address pair on one line and without any email addresses or phone numbers. Also, `\thanks` is available under transmag journal mode even though the names are entered much like the long format under conference mode. See the file `bare_jrnl_transmag.tex` for an example of author entry under transmag mode.

4.3 Running Headings

The running headings are declared with the `\markboth{}{}` command. The first argument contains the journal name information and the second contains the author name and paper title. For example:

```
\markboth{Journal of Quantum Telecommunications,~Vol
.~1, No.~1,~January~2025}{Shell \MakeLowercase{\text
it{et al.}}: A Novel Tin Can Link}
```

Note that because the text in the running headings is automatically capitalized, the `\MakeLowercase{}` command must be used to obtain lower case text. The second argument is used as a page heading only for the odd number pages after the title page for two sided (duplex) journal papers. This page is such an example. Technote papers do not utilize the second argument. Conference papers do not have running headings, so `\markboth{}{}` has no effect when in conference mode. Authors should not put any name information in the headings (if used) of anonymous peer review papers.

4.4 Publication ID Marks

Publication ID marks can be placed on the title page of journal and technote papers via the `\IEEEpubid{}` command:

```
\IEEEpubid{0000--0000/00\$00.00~\copyright~2015 IEEE
}
```

Although authors do not yet have a valid publication ID at the time of paper submission, `\IEEEpubid{}` is useful because it provides a means to see how much of the title page text area will be unavailable in the final publication. This is especially important in technote papers because, in some journals, the publication ID space can consume more than one text line. If `\IEEEpubid{}` is used, a second command, `\IEEEpubidadjcol` must be issued somewhere in the *second* column of the title page. This is needed because LaTeX resets the text height at the beginning of each column. `\IEEEpubidadjcol` "pulls up" the text in the second column to prevent it from blindly running into the publication ID.

Publication IDs are not to be placed by the author on camera ready conference papers so `\IEEEpubid{}` is disabled in conference mode. Instead the bottom margin is automatically increased by IEEEtran when in conference mode to give the IEEE room for such marks at the time of publication. In draft mode, the publisher ID mark will *not* be printed at the bottom of the titlepage, but room will be cleared for it.

Publication ID marks are perhaps less important with compsoc papers because IEEE Computer Society journals place the publisher ID marks within the bottom margin so as not to affect the amount of page space available for text.

4.5 Special Paper Notices

Special paper notices, such as for invited papers, can be declared with:

```
\IEEEspecialpapernotice{(Invited Paper)}
```

Special paper notices in journal and technote papers appear between the author names and the main text. The title page of this document has an example. For conference papers, the special paper notice is placed between the title and the author names.

Much more rarely, there is sometimes a need to gain access to the space across both columns just above the main text. For instance, a paper may have a dedication [15]. IEEEtran provides the command `\IEEEaftertitletext{}` which can be used to insert text or to alter the spacing between the title area and the main text:

```
\IEEEaftertitletext{\vspace{-1\baselineskip}}
```

Authors should be aware that IEEEtran carefully calculates the spacing between the title area and main text to ensure that the main text height of the first page always is equal to an integer number of normal sized lines (unless the top or bottom margins have been overridden by CLASSINPUTs). Failure to do this can result in underfull vbox errors and paragraphs being "pulled apart" in the second column of the first page if there isn't any rubber lengths (such as those around section headings) in that column. The contents of `\IEEEaftertitletext{}` are intentionally allowed to bypass this "dynamically determined title spacing" mechanism, so authors may have to manually tweak the height (by a few points) of the `\IEEEaftertitletext{}` contents (if used) to avoid an underfull vbox warning.

5 Abstract and Index Terms

The abstract is generally the first part of a paper after `\maketitle`. The abstract text is placed within the abstract environment:

```
\begin{abstract}
We propose ...
\end{abstract}
```

Math, special symbols and/or citations should generally not be used in abstracts.[4]

Journal and technote papers also have a list of key words (index terms) which can be declared with:

```
\begin{IEEEkeywords}
Broad band networks, quality of service, WDM.
\end{IEEEkeywords}
```

To obtain a list of valid keywords from the IEEE, just send a blank email to keywords@ieee.org. A list of IEEE Computer Society approved keywords can be obtained at http://www.computer.org/mc/keywords/keywords.htm. Do not use math or special symbols in the keywords.

The IEEE Computer Society and IEEE TRANSACTIONS ON MAGNETICS formats present a difficulty in that compsoc and transmag journal (but *not* compsoc conference) papers place the abstract and index terms sections in single column format just below the author names, but the other IEEE formats place them in the first column of the main text before the first section. To handle this, IEEEtran offers a command, `\IEEEtitleabstractindextext`, that is to be declared *before* `\maketitle`, and whose single argument holds the text/sections that are to appear in single column format after the author names:

```
\IEEEtitleabstractindextext{%
\begin{abstract}
We propose ...
\end{abstract}
\begin{IEEEkeywords}
Broad band networks, quality of service, WDM.
\end{IEEEkeywords}}
```

To facilitate dual compilation, IEEEtran provides another command, `\IEEEdisplaynontitleabstractindextext`, which will "become" whatever was declared in `\IEEEtitleabstractindextext` when in non-compsoc, non-transmag or conference mode (as compsoc conferences use the same placement for the abstract and index terms as traditional conferences do). That is to say, the abstract and index terms sections can be automatically "teleported' to the appropriate place they need to be depending on the document mode. `\IEEEdisplaynontitleabstractindextext` should typically be placed just after `\maketitle` (and before `\IEEEpeerreviewmaketitle` if used).

4. That said, if it is permitted or required, be aware that in order to preserve the distinction between constructs such as vector and scalar forms, IEEEtran defaults to using non-bold math within the abstract. However, bold math better matches the bold text font used for the abstract text. If a bold math font is desired, just issue a `\boldmath` command at the start of the abstract.

6 Sections

Sections and their headings are declared in the usual LATEX fashion via `\section`, `\subsection`, `\subsubsection`, and `\paragraph`. In the non-compsoc modes, the numbering for these sections is in upper case Roman numerals, upper case letters, Arabic numerals and lower case letters, respectively. In compsoc mode, Arabic numerals are used exclusively for (sub)section numbering.

The `\paragraph` section is not allowed for technotes or compsoc conferences as these generally are not permitted to have such a deep section nesting depth. If needed, `\paragraph` can be restored by issuing the command `\setcounter{secnumdepth}{4}` in the document preamble.

Note that IEEE Computer Society journals (but not conferences!) are unusual in that they raise the very first section (the introduction) heading above the start of the text. IEEEtran provides a command to produce this effect:

```
\IEEEraisesectionheading{\section{Introduction}\label{sec:introduction}}
```

This command is not intended for any use other than the introduction section in compsoc journal mode. Note the need to keep any `\label` that is to refer to the section immediately after `\section` in the above as `\IEEEraisesectionheading` puts `\section` within a raised box.

6.1 Initial Drop Cap Letter

The first letter of a journal paper is a large, capital, oversized letter which descends one line below the baseline. Such a letter is called a "drop cap" letter. The other letters in the first word are rendered in upper case. This effect can be accurately produced using the IEEEtran command `\IEEEPARstart{}{}`. The first argument is the first letter of the first word, the second argument contains the remaining letters of the first word. The drop cap of this document was produced with:

```
\IEEEPARstart{W}{ith}
```

Note that some journals will also render the second word in upper case—especially if the first word is very short. For more usage examples, see the `bare_jrnl.tex` example file.

7 Citations

Citations are made with the `\cite` command as usual. IEEEtran will produce citation numbers that are individually bracketed in IEEE style. ("[1], [5]" as opposed to the more common "[1, 5]" form.) The base IEEEtran does not sort or produce compressed "ranges" when there are three

or more adjacent citation numbers. However, IEEEtran predefines some format control macros to facilitate easy use with Donald Arseneau's cite.sty package [16]. So, all an author has to do is to call cite.sty:

```
\usepackage{cite}
```

and the adjacent citation numbers will automatically be sorted and compressed (ranged) IEEE style. (Of course, multiple adjacent citations should always all be declared within a single `\cite`, comma separated, for this to work.) Invoke cite.sty's noadjust option to prevent an unwanted leading space from occurring should a citation ever need to be enclosed in parenthesis.

One complication in compsoc mode is that the IEEE Computer Society does not compress, but does sort, adjacent citation numbers. Version 4.0 and later of cite.sty provides a nocompress option that disables compression, but preserves sorting. Thus,

```
\ifCLASSOPTIONcompsoc
  % requires cite.sty v4.0 or later (November 2003)
  \usepackage[nocompress]{cite}
\else
  \usepackage{cite}
\fi
```

can be used with universal applicability.

Note that, if needed (e.g., next to a non-punctuation, non-space character), cite.sty's `\cite` command will automatically add a leading space. i.e., "(`\cite{mshell01}`)" will become like "([1])". If this behavior is not desired, use the cite package's noadjust option (cite.sty V3.8 and later) which will turn off the added spaces:

```
\usepackage[noadjust]{cite}
```

`\cite` also allows for an optional note (e.g., `\cite[Th. 7.1]{mshell01}`). If the `\cite` with note has more than one reference, the note will be applied to the last of the listed references. It is generally desirable that if a note is given, only one reference should be listed in that `\cite`.

8 Equations

Equations are created using the traditional `equation` environment:

```
\begin{equation}
\label{eqn_example}
x = \sum\limits_{i=0}^{z} 2^{i}Q
\end{equation}
```

which yields

$$x = \sum_{i=0}^{z} 2^{i} Q. \tag{1}$$

Use the `displaymath` environment instead if no equation number is desired. When referring to equations, articles in IEEE publications do not typically use the word "equation," but rather just enclose the equation number in parentheses, e.g.,

```
... as can be seen in (\ref{eqn_example}).
```

IEEE's two column format puts serious constraints on how wide an equation can be. So, a fair portion of the effort in formatting equations usually has to be devoted to properly breaking them. It is the author's responsibility to ensure that all equations fit into the given column width. In rare circumstances, it is possible to have a few equations that span both columns (see Section 10.4.1), but the vast majority of over-length equations have to be broken across multiple lines.

TABLE 1
Math Spacings Used by LaTeX

Size	Width	Cmd.	Used for	Example
small	1/6 em	`\,`	symbols	$a\,b$
medium	2/9 em	`\:`	binary operators	$a+b$
large	5/18 em	`\;`	relational operators	$a=b$
negative small	−1/6 em	`\!`	misc. uses	$a\!b$

9 Multi-line Equations

Perhaps the most convenient and popular way to produce multiline equations is LaTeX 2ε's `eqnarray` environment. However, `eqnarray` has several serious shortcomings:

1) the use of 2×`\arraycolsep` for a column separation space does not provide natural math spacing in the default configuration;
2) column definitions cannot be altered;
3) it is limited to three alignment columns;
4) column alignment cannot be overridden within individual cells.

There are a number of vastly superior packages for formatting multiline mathematics. Perhaps the most popular is the amsmath package [11]. Amsmath is a comprehensive work which contains many helpful tools besides enhanced multiline alignment environments. So, all authors should give serious consideration to its use—regardless of what they use to generate aligned equations. One thing to be aware of is that, upon loading, amsmath will configure LaTeX to disallow page breaks within multiline equations (even within non-amsmath defined environments). The philosophy here is that author should manually insert breaks where desired so as to ensure that breaks occur only at acceptable points. To restore IEEEtran's ability to automatically break within multiline equations, load amsmath like:

```
\usepackage{amsmath}
\interdisplaylinepenalty=2500
```

Another extremely powerful set of alignment tools, one of which is a totally rewritten `eqnarray` environment, is provided by mathenv.sty which is part of Mark Wooding's MDW Tools [17].

Finally, IEEEtran provides a fully integrated custom IEEEeqnarray family of commands (see Appendix F) that are designed to have almost universal applicability for many different types of alignment situations.

Nevertheless, it is instructive to show a simple example using the standard `eqnarray` in order to explain some of the fine points of math spacing under LaTeX. As shown in Table 1, TeX normally draws from four different spacings when typesetting mathematics. In order to produce precise (and

correct) mathematical alignments, it is crucial to understand how to control such spacing. Consider a multiline equation

$$\begin{aligned} Z &= x_1 + x_2 + x_3 + x_4 + x_5 + x_6 \\ &\quad +a + b && (1)\\ &\quad +a + b && (2)\\ &\quad + a + b && (3)\\ &\quad + a + b && (4) \end{aligned}$$

(in typical IEEE style) which was produced by

```
\setlength{\arraycolsep}{0.0em}
\begin{eqnarray}
Z&{}={}&x_1 + x_2 + x_3 + x_4 + x_5 + x_6\nonumber\\
&&+a + b\\
&&+{}a + b\\
&&{}+a + b\\
&&{+}\:a + b
\end{eqnarray}
\setlength{\arraycolsep}{5pt}
```

Lines one through four show some possible ways the $+a+b$ line could be implemented.[5] Only number four is the correct way for most IEEE purposes. In TEX's math mode, spacing around operators can be inhibited by enclosing them within braces (e.g., `{=}`) or forced by surrounding them with "empty ords" (e.g., `{}={}`). It is important to understand that the empty ords do not have width themselves. However, their presence causes TEX to place space around the operators as if they were "next to something." With this in mind, the first step in the example is to set `\arraycolsep` to zero to prevent `eqnarray` from putting in the unwanted, artificial, inter-column spacing. Placing empty ords around the equal sign then forces the correct natural spacing. Alternatively, `\arraycolsep` could have been set to 0.14 em and the empty ords around the equal sign eliminated.[6] It is important to remember to restore `\arraycolsep` to its default value of 5 pt after the `eqnarray` is complete as other environments (such as `array`) depend on it. (Alternatively, the structure could have been enclosed in a group of braces to keep the change local—which has the added advantage of not requiring that the user remember what the correct default value is.)

The first line is incorrect because a is being indicated as a positive quantity rather than something that must be added to the previous line. (i.e., the $+$ is being treated as a unary, rather than a binary, operator.) In line two, adding an empty ord to the right side of the plus sign does nothing, except to demonstrate that empty ords have zero width. Adding an empty ord to the left side of the plus sign (line three) does engage binary spacing, but causes an unwanted[7] right shift of the line. Finally, manually adding a medium space to the right side only of the plus sign in line four does the trick. The suppression of automatic spacing around the plus sign (`{+}`) is unneeded in this case, but may be required in other alignment environments that "expand" such operators by default.

5. In this example, the equation numbering system is (ab)used to identify lines.

6. This assumes that 1 em in the text font has the same width as 1 em in the math font. For the standard fonts, this is indeed the case.

7. The IEEE normally wants all of the lines left aligned, but there are cases when such an indention may be desirable.

Another way around the spacing problem is to use only two alignment columns (as is done by amsmath.sty's `\align`). e.g., in the previous example, "$Z =$" would be contained in the first column.

9.1 Cases Structures

Incidentally, the `numcases` (or `subnumcases`) environments in Donald Arseneau's cases.sty package [18] should be used when "cases" structures in which each branch can be referenced with a different equation (or subequation) number are needed:

$$|x| = \begin{cases} x, & \text{for } x \geq 0 \quad (5a)\\ -x, & \text{for } x < 0 \quad (5b) \end{cases}$$

because those built from the `array` or amsmath `cases` environments will have a single equation number that encompasses both branches.

Be aware that if amsmath (which will be automatically loaded under comsoc mode if the user did not do so) is to be used with cases.sty, the latter should be loaded after the former or else an error "`Command \subequations already defined`" may occur.

10 Floating Structures

Authors should keep in mind when choosing an appropriate optional placement argument for the figure/table environments that most IEEE journals strongly favor the positioning of floats to the top of the page and rarely, if ever, use bottom floats. IEEE Computer Society journals also favor top floats, but do occasionally employ bottom floats. Furthermore, IEEE journals never place floats in the first column of the first page and rarely (if ever) do they do so in the second column of the first page. Middle in-text placement ("here") is usually not used for IEEE work with one notable exception—IEEE Computer Society conferences.

Note that LATEX 2ε's float routine places footnotes above bottom floats. To change this so that footnotes appear below the bottom floats (as the IEEE does) invoke the `\fnbelowfloat` command provided by Sigitas Tolušis' stfloats package [19] (see Section 10.4 for more features of the stfloats package).

10.1 Figures

Figures handled in the standard LATEX manner. For example:

```
\begin{figure}[!t]
\centering
\includegraphics[width=2.5in]{myfigure}
\caption{Simulation results for the network.}
\label{fig_sim}
\end{figure}
```

Note that (1) figures should be centered via the LATEX `\centering` command—this is a better approach than using the `center` environment which adds unwanted vertical spacing; (2) the caption follows the graphic; and (3) any labels must be declared *after* (or within) the caption command.

When referring to figures in typical IEEE papers, authors should use the abbreviation "Fig.", but in IEEE Computer Society *conference* papers they should use the full word "Figure". IEEEtran provides the string macro `\figurena`

me which contains the correct name to use for the given formatting mode.

The \includegraphics command is the modern, preferred, way of including images and provides a flexible interface that makes it easy to scale graphics to size. To use it, the graphics or graphicx (the latter is recommended) must first be loaded.

It is strongly recommended that authors be familiar with the graphics package documentation [20] as well as Keith Reckdahl's excellent *Using Imported Graphics in LaTeX 2ε* [21]. The reader is reminded that the "draftcls" or "draftclsnofoot", not "draft", class option must be selected in order to get draft papers with visible figures.

As explained in Appendix D, Encapsulated PostScript (EPS) or Portable Document Format (PDF) is the preferred graphics format for LaTeX work. Furthermore, the user's drawing/graphing application should be capable of outputing directly in EPS (or PDF) vector form (which will not degrade or pixelize when magnified)—although photos will likely have to be in (EPS/PDF/JPEG/PNG) bitmap form. Be aware that the use of pdfLaTeX is required for image formats other than EPS.

The psfrag package [22] might also be of interest. Psfrag allows the user to "go into" an EPS graphic and replace text strings contained in it with real LaTeX code. In this manner, LaTeX's extensive support of mathematical symbols and fonts can be extended to figures made with applications with more modest glyph support. Using psfrag does require the use of the dvips DVI to PostScript conversion step (not pdfLaTeX's PDF mode) as some of the features of the PostScript language have to be utilized.[8] pdfLaTeX users can use psfrag by "preprocessing" their figures by importing them into a dummy document using psfrag, running LaTeX followed by dvips, then converting the PostScript output to a PDF graphic for direct importation into the main document which is then processed by pdfLaTeX. There is additional usage information on psfrag in the *Using Imported Graphics in LaTeX 2ε* guide [21].

10.1.1 Subfigures

Subfigures can be obtain via the use of Steven Douglas Cochran's subfigure [23] or subfig [24] packages. Be forewarned that the former is no longer being maintained and, although self-contained and compatible with IEEEtran, is becoming incompatible with an increasing number of other LaTeX packages including fixltx2e.sty. For this reason, subfigure.sty is not recommended for new work and will not be covered here.

It is important to note that subfig.sty package options are usually required to obtain IEEE compliant subfigure captions. Furthermore, compsoc format requires a larger sans serif font than the serif footnote size font used in traditional IEEE formatting. There is a further complication with subfig.sty in that this package depends on caption.sty, which, in its default configuration, will overrride IEEEtran's handling of captions—resulting in non-IEEE style main captions. To prevent this, be sure to invoke subfig.sty's caption=false option, which has been available since version 1.3 (2005/06/28). Thus, the recommended way to load subfig.sty is:

```
\ifCLASSOPTIONcompsoc
  \usepackage[caption=false,font=normalsize,labelfon
t=sf,textfont=sf]{subfig}
\else
  \usepackage[caption=false,font=footnotesize]{subfi
g}
\fi
```

Because multiple subfigures usually require more width than is available in a single column, they are often used within the double column figure environment (Section 10.4):

```
\begin{figure*}[!t]
\centering
\subfloat[Case I]{\includegraphics[width=2.5in]{subf
igcase1}
\label{fig_first_case}}
\hfil
\subfloat[Case II]{\includegraphics[width=2.5in]{sub
figcase2}
\label{fig_second_case}}
\caption{Simulation results for the network.}
\label{fig_sim}
\end{figure*}
```

Note how captions can be tagged to each of the subfigures as well as to the overall figure via an optional argument to the \subfloat command. However, most IEEE authors/journals do not employ subfigure captions, but instead reference/describe all of the subfigures (a), (b), etc., within the main caption. Be aware that for subfig.sty to generate the (a), (b), etc., subfigure labels the optional argument to \subfloat must be present. If a subcaption is not desired, just leave its contents blank (e.g., \subfloat[]). \hfil is used as a subfigure separator to achieve equal spacing around the graphics. More complex implementations are possible. Note that the total width of all the subfigures on a line must be less than the text width or else an unwanted line break will occur. Multiple lines of subfigures can be used within a figure if needed. See the subfig.sty documentation as well as the *Using Imported Graphics in LaTeX 2ε* guide [21] for more details.

Axel Sommerfeldt's modern and actively maintained subcaption.sty package [25] can not be recommended at this time because it does not provide an option to prevent the underlying caption.sty from taking control of main caption formatting away from IEEEtran.

10.2 Algorithms

IEEE publications use the figure environment to contain algorithms that are not to be a part of the main text flow. Peter Williams' and Rogerio Brito's algorithmic.sty package [26] or Szász János' algorithmicx.sty package [27] (the latter is designed to be more customizable than the former) may be of help in producing algorithm-like structures (although authors are of course free to use whatever LaTeX commands they are most comfortable with in this regard). However, do *not* use the floating algorithm environment of algorithm.sty (also by Williams and Brito) or algorithm2e.sty (by Christophe Fiorio) as the only floating structures IEEE uses are figures and tables. Furthermore, IEEEtran will not be in control of the (non-IEEE) caption style produced by the algorithm.sty or algorithm2e.sty float environments.

8. PDF is much like a subset of PostScript—the latter is a Turing complete programming language, the former is not.

TABLE 2
A Simple Example Table

First	Next
1.0	2.0

10.3 Tables

Tables are handled in a similar fashion, but with a few notable differences. For example, the code

```
\begin{table}[!t]
\renewcommand{\arraystretch}{1.3}
\caption{A Simple Example Table}
\label{table_example}
\centering
\begin{tabular}{c||c}
\hline
\bfseries First & \bfseries Next\\
\hline\hline
1.0 & 2.0\\
\hline
\end{tabular}
\end{table}
```

results in Table 2. Note that the IEEE places table captions *before* the tables and, given that they serve much like titles, are usually capitalized except for words such as a, an, and, as, at, but, by, for, in, nor, of, on, or, the, to and up, which are usually not capitalized unless they are the first or last word of the caption.

Be aware that, to prevent a change of meaning that would result from case changes, the IEEE generally uses the standard text font, not the small caps font, when rendering units as well as letters in math in table captions. This can be achieved via the use of `\upshape`:

```
\caption{Diagnosis of Rotor Faults in a DRFOC Drive U
sing the VCT(Flux Loop Bandwidth (FLB) = 10 {\upshap
e Hz}; 75% Load; 1450 {\upshape r/min})}
```

Thanks to Zhaowen Hou for providing information on this topic as well as the above example.

Within the table environment, the default text size is footnotesize which is what IEEE typically uses for tables. When using the tabular environment to construct tables, it is usually a good idea to increase the value of `\arraystretch` above unity to "open up" the table rows a tad. Also, IEEE often uses tables with "open sides," (without vertical lines along each side) although the "closed side" form (e.g., Table 1) is more commonly used for the tables within this document.

Unfortunately, the standard LaTeX 2$_\varepsilon$ tabular environment has a number of shortcomings. Two notable problems are (1) the corners where lines meet are improperly formed; and (2) it is not very flexible in terms of user control. For these reasons, authors are urged to look into some of the other packages for making tables. A good one that provides revised "drop-in replacements" for both the tabular and array environments is Frank Mittelbach's and David Carlisle's array package [28]. Even more powerful (and complex) is the tabular and array environments provided by the mdwtab.sty package which is part of Mark Wooding's MDW Tools [17].

TABLE 3
The Skewing Angles (β) for $\mathrm{Mu(H)} + \mathrm{X_2}$ and $\mathrm{Mu(H)} + \mathrm{HX}$ [a]

	$\mathrm{H(Mu)} + \mathrm{F_2}$	$\mathrm{H(Mu)} + \mathrm{Cl_2}$
β(H)	80.9°[b]	83.2°
β(Mu)	86.7°	87.7°

[a] for the abstraction reaction, $\mathrm{Mu} + \mathrm{HX} \rightarrow \mathrm{MuH} + \mathrm{X}$.
[b] 1 degree $= \pi/180$ radians.

As an alternative, IEEEtran offers the IEEEeqnarraybox command which can also be used to produce tables[9] of high quality. See Appendix F for more details.

10.3.1 Footnotes Within Tables

Footnotes normally cannot be placed directly within some commands and environments such as `\parbox`, `tabular`, etc., because they become "trapped" inside. One way around this is to split the place the footnote marker (`\footnotemark`) is located (within the table) from where the footnote text itself is declared (outside of the table using `\footnotetext`).

Another approach is to use the footnote.sty package (which is part of Mark Wooding's MDW Tools [17]) which allows environments to be configured so as not to trap footnotes:

```
\usepackage{footnote}
\makesavenoteenv{tabular}
```

Note that is probably not a good idea to use footnotes in floating structures (like `table`) because the position of each can move relative to one another. To put the footnote at the end of a table instead of at the bottom of the page, just enclose `tabular`, etc., inside a minipage (no footnote package needed). A very good approach for handling footnotes within tables (including those that float) is to use Donald Arseneau's threeparttable package [29] which was used to generate Table 3 (the code of which is an example in the threeparttable.sty file).

10.4 Double Column Floats

LaTeX's `figure*` and `table*` environments produce figures and tables that span both columns. This capability is sometimes needed for structures that are too wide for a single column.

It is a limitation of the LaTeX 2$_\varepsilon$ kernel that double column floats cannot be placed at the bottom of pages. That is to say "`\begin{figure*}[!b]`" will not normally work as intended. Authors that need this capability should obtain and load Sigitas Tolušis' stfloats package [19] which patches the LaTeX 2$_\varepsilon$ output routine to allow it to handle double column floats at the bottom of pages. Please note that stfloats is a very invasive package which may not work with versions of LaTeX other than the standard LaTeX 2$_\varepsilon$ release and may cause problems with other packages that modify the output and/or float routines (such as those that balance columns, alter the placement of floating figures, etc.). IEEE authors are warned *not* to use packages that allow material

9. Table 1 was made using this command.

to be placed across the middle of the two text columns (such as cuted.sty, midfloat.sty, etc.) as the IEEE does not do this.

Another LaTeX 2_ε limitation (patched with stfloats or not) is that double column floats will not appear on the same page where they are defined. So, the user will have to define such things prior to the page on which they are to (possibly) appear.

LaTeX 2_ε (patched with stfloats or not) does not attempt to keep double and single column floats in sequence with each other. This can be fixed by loading Frank Mittelbach, David Carlisle and Chris Rowley's fixltx2e package (already installed on most LaTeX systems) [30]. Note that fixltx2e.sty is the replacement (and superset) of the older fix2col.sty [30]. However, fixltx2e/fix2col should not be used with the stfloats package as they both modify some of the same float routines in different ways.

Be aware that LaTeX 2_ε kernels dated 2015 and later have fixltx2e.sty's corrections already built into the system in which case a warning will be issued if an attempt is made to load fixltx2e.sty as it is no longer needed.

Morten Høgholm's dblfloatfix package [31] provides the combined functionality of both the fixltx2e and stfloats packages and is now the recommended way to obtain these features.

Finally, authors should also be aware that the LaTeX 2_ε kernel (patched with stfloats or not) has a long standing limitation in that it will not allow rubber space that spans both columns to stretch or shrink as needed for each of the two main text columns. Therefore, it is possible for double column floats to cause underfull vbox errors because the remaining text height may not be equal to an integer number of normal size lines. The problem can occur in main text columns (on pages with double column floats) that do not have vertical rubber spacing (such as that around section headings, equations, etc.) and results in underfull vbox warnings coupled with paragraphs that are "pulled apart" from each other. To correct this, users can manually tweak the amount of space between the double column structure and main text by inserting a command like

```
\vspace*{-3pt}
```

(adjusted as needed) within the double column structure. Incidentally, IEEEtran automatically compensates for this problem when forming the paper title.

10.4.1 Double Column Equations

It is possible, but not pleasant, to use `figure*` to obtain double column equations. The IEEE rarely uses double column equations because they can waste space, so this capability is easy to abuse. Authors who are considering the use of a double column equation should verify that there are a few examples of such in papers previously published in the journal they plan to submit to.

There are complications. Although the IEEE does not place constraints on the order of the double column equations relative to the equations of the main text (that is to say a set of double column equations can be at the top or bottom of a page in which they would normally appear in the middle had they been regular equations), the double column equation numbers must increase as one progresses down the page (i.e., double column equations at the bottom of a page must be of higher number than those at the top). Furthermore, double column equations should appear on the same page where they are referenced (on the page they would have appeared had they been regular equations). Compounding the difficulty even further is the fact that LaTeX 2_ε will not place double column equations on the same page on which they are defined. Finally, the IEEE does not generally allow other figures or tables to come between the double column equations and the main text (which are separated from each other by a rule). All of this means that the place where a double column equation must be defined has to be "disconnected" from the place where it will eventually be referred to in the text—and the user will have to manually intervene in the equation numbering system.

Therefore, users have to (1) define double column equations on the page *prior* to the one that they are to appear; (2) reset the equation counter when the double column equations are defined so as not to disturb the regular equation numbers; (3) manually set the double column equation numbers and (4) increment the equation counter at the point the double column equations are referenced in the text so that they are accounted for in the numbering of the regular equations after that point.

To do all of this, it is convenient to have a "scratch pad" counter to temporarily save equation numbers. This can be done via a command such as

```
\newcounter{MYtempeqncnt}
```

in the preamble of the document. Now, the double column equations are defined on the page *prior* to the one in which they are to appear (and in this example supposed that they are to be equation numbers six and seven):

```
\begin{figure*}[!t]
% ensure that we have normalsize text
\normalsize
% Store the current equation number.
\setcounter{MYtempeqncnt}{\value{equation}}
% Set the equation number to one less than the one
% desired for the first equation here.
% The value here will have to changed if equations
% are added or removed prior to the place these
% equations are referenced in the main text.
\setcounter{equation}{5}
\begin{equation}
\label{eqn_dbl_x}
x = 5 + 7 + 9 + 11 + 13 + 15 + 17 + 19 + 21+ 23 + 25
 + 27 + 29 + 31
\end{equation}
\begin{equation}
\label{eqn_dbl_y}
y = 4 + 6 + 8 + 10 + 12 + 14 + 16 + 18 + 20+ 22 + 24
 + 26 + 28 + 30
\end{equation}
% Restore the current equation number.
\setcounter{equation}{\value{MYtempeqncnt}}
% The IEEE uses as a separator
\hrulefill
% The spacer can be tweaked to stop underfull vboxes.
\vspace*{4pt}
\end{figure*}
```

The result of which is shown at the top of the next page. This technique allows the definition of the equations to be positioned arbitrarily as needed so that the (floating) equations will appear where desired. The "[!t]" option forces LaTeX to do its best to place the equations at the top of the

$$x = 5 + 7 + 9 + 11 + 13 + 15 + 17 + 19 + 21 + 23 + 25 + 27 + 29 + 31 \quad (6)$$

$$y = 4 + 6 + 8 + 10 + 12 + 14 + 16 + 18 + 20 + 22 + 24 + 26 + 28 + 30 \quad (7)$$

next page. Had it been "[!b]" instead, then the stfloats (or even better, dblfloatfix) package would need to be loaded and the `\vspace` command, followed by the `\hrulefill` command, would have to occur *before* the equations in the figure.

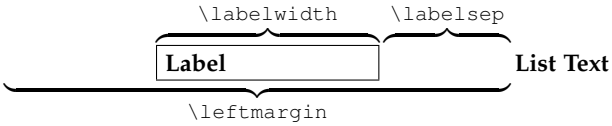
The double column equations can then been referenced in the main text like:

```
% The previous equation was number five.
% Account for the double column equations here.
\addtocounter{equation}{2}
As can be seen in (\ref{eqn_dbl_x}) and
(\ref{eqn_dbl_y}) at the top of the page ...
```

Thankfully, double column equations are rare.

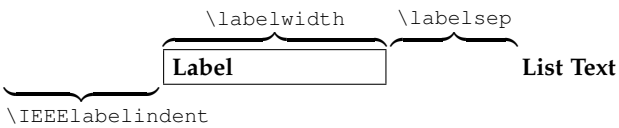
11 LISTS

The traditional LaTeX itemize, enumerate and description (IED) list environments are ill-suited for producing the style of lists used in IEEE publications. The main problem is that they do not provide the user a means for controlling the parameters of the resultant list. Furthermore, making global changes to the parameters of the underlying `\list` will result (often unexpectedly to a user) in the improper behavior of other commands that depend on it, such as `\quote`. Finally, LaTeX's `\list` considers the left margin of the list text to be the reference point that determines how the list is positioned relative to the left margin of the main text:



This contrasts with IEEE lists which use the label box as the reference point for the list structure. i.e., for a given circumstance, the list labels will be indented by a certain amount, the list text block will be indented from the label boxes by a given amount and these spacings will determine the position of the list text.

For these reasons, IEEEtran provides enhanced IED list environments that make it much easier to produce IEEE style lists. The underlying `\list` remains the same as in traditional LaTeX so as not to break code that depends upon it. IEEEtran uses a new length variable, `\IEEElabelindent`, so that users can specify IED list structures directly in IEEE fashion:



The IEEEtran IED lists ignore all "external" changes to the list length parameters. Instead, IED lists are controlled exclusively via two interfaces:

1) "global" control via the `\IEEEiedlistdecl` command; and
2) "local" control via an *optional* argument that can be provided to `\itemize`, `\enumerate`, and `\description`.

For example, declaring

```
\renewcommand{\IEEEiedlistdecl}{\settowidth{\labelwidth}{Hello}}
```

in an IEEEtran document will set the default width of the label boxes in *all* later IED lists to be equal to the width of "Hello". Note: Because setting a `\labelwidth` is so commonly performed, IEEEtran provides a command: `\IEEEsetlabelwidth{X}` which is a shorter form of: `\settowidth{\labelwidth}{X}`.

The local control is used if the parameters are to apply only to an individual IED list:

```
\begin{itemize}[\IEEEsetlabelwidth{$\gamma$}]
```

Within an IED list, the local control is executed just after the global control and therefore, the commands in the local control can both augment and countermand those in the global control. Please note that the code in the local and global controls are executed in the same manner as normal LaTeX code. Therefore, the user should ensure that unwanted blank spaces do not appear in the controls. If a control definition is too long to fit on one line, shield the end of lines with "%" to prevent them from being interpreted as blanks (Section 4.2.1 has some information on this topic). Also, note that the LaTeX parser requires that braces be placed around commands with optional arguments that are placed directly within the optional arguments of other commands:

```
\begin{itemize}[{\mycmd[1]{example}}]
```

This IEEEtran IED implementation makes it easy to control IED lists, even when they are deeply nested.

The default spacings the IED lists use are stored in various length (not macro) commands. Changes to these "master" defaults are rarely needed and should be done only at the beginning of the document, *not in the IED list controls*. These constants will now be briefly explained.

`\IEEEilabelindent`: This length is the default amount the itemized list label boxes are indented from the left margin. The IEEE seems to use at least two different values. For example, in the IEEE/OSA JOURNAL OF LIGHTWAVE TECHNOLOGY and the IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, they tend to use an indention equal to `\parindent`, while for IEEE TRANSACTIONS ON COMMUNICATIONS they tend to indent itemized lists a little more (`1.3\parindent`). The shorter length is stored as `\IEEEilabelindentA` and the longer as `\IEEEilabelindentB`. The default is to use the shorter version. To use the longer version do a

```
\setlength{\IEEEilabelindent}{\IEEEilabelindentB}
```

at the beginning of the document.

\IEEEelabelindent: This length is the default amount the enumerated list label boxes are indented from the left margin. Normally, the same as \parindent.

\IEEEdlabelindent: Ditto for description list labels. Normally, the same as \parindent.

\IEEEiednormlabelsep: This length is the normal default spacing between the IED list label boxes and the list text.

\IEEEiedmathlabelsep: For nomenclature description lists (a list of math symbols and their explanations), the IEEE usually increases the separation between the terms and the definitions. This length is set to the longer than normal length. To invoke its use, just issue the command \IEEEusemathlabelsep in a list control.

\IEEEiedtopsep: This length is the extra vertical separation put above and below each IED list. The IEEE usually puts a little extra spacing around each list. However, this extra spacing is barely noticeable.

\IEEElabelindentfactori through **\IEEElabelindentfactorvi**: These contain the factors by which the effective \IEEElabelindent is reduced as the list nesting depth increases. The IEEE normally decreases the amount of indention as the list nesting level increases because there isn't much room to indent with two column text. IEEEtran has an "automatic indention cut-back" feature that provides this behavior. The actual amount the label boxes will be indented is \IEEElabelindent multiplied by the \IEEElabelindentfactorX corresponding to the level of nesting depth (where "X" is the nesting depth in roman numerals). This provides a means by which the user can alter the effective \IEEElabelindent for deeper levels. There may not be such a thing as correct "standard IEEE" values. What the IEEE actually does may depend on the specific circumstances. The first list level almost always has full indention. The second levels usually have only 75% of the normal indentation. Third level and greater nestings are very rare, and probably don't use any indentation. These factors are not lengths, but rather constant macros like \baselinestretch so \renewcommand should be used if they need to be changed. The default values are

```
\IEEElabelindentfactori   1.0
\IEEElabelindentfactorii  0.75
\IEEElabelindentfactoriii 0.0
\IEEElabelindentfactoriv  0.0
\IEEElabelindentfactorv   0.0
\IEEElabelindentfactorvi  0.0
```

The use of these factors in IED lists may be suspended by issuing the command \IEEEnolabelindentfactortrue in a list control (which has the same effect as setting all the indent factors to 1.0).

Normally, IEEEtran automatically calculates \leftmargin based upon the current values of \IEEElabelindent, \labelwidth and \labelsep. To stop this auto-calculation so that a manually specified value of \leftmargin is used instead, just use \IEEEnocalcleftmargintrue in a list control. This feature should not be needed during the course of normal IEEE related work.

IEEEtran provides a means to manually specify the justification within the IED list label boxes. The commands \IEEEiedlabeljustifyl, \IEEEiedlabeljustifyc and \IEEEiedlabeljustifyr can be used in a list control to justify the list labels to the left, center, and right sides, respectively. Itemize and enumerate lists automatically default to right justification, while description defaults to left justification. The justification commands should not be needed during the course of normal IEEE related work.

In addition to modifying the behavior of itemize, enumerate and description, IEEEtran also provides the respective aliases IEEEitemize, IEEEenumerate and IEEEdescription, which provides a way for the user to access the IEEE style list environments even in the event another package is loaded that overrides the IED list environments. For specialized applications, the original LaTeX IED list environments are retained as LaTeXitemize, LaTeXenumerate and LaTeXdescription.

11.1 Itemize

The itemized lists will normally automatically calculate the width of whatever symbol the current list level is using so that a user can just call \begin{itemize}...\end{itemize} without doing anything special. Furthermore, the auto-label-width feature will work properly even if \labelitemX has been redefined (where "X" indicates "i,ii, .. iv", whichever is appropriate) *before* the list begins. However, if any item symbols are to be specified via \item[X] (this is rare and may well be nonstandard as far as IEEE related work is concerned), then the following form can be used:

```
\begin{itemize}[\IEEEsetlabelwidth{Z}]
\item[X] blah
\item[Y] blah
.
.
\end{itemize}
```

where "Z" is the longest label in the list.

11.2 Enumerate

The important thing to note about enumerated lists is that the \labelwidth will default to the length of "9)" in the normal size and style. Therefore, the width of the longest label will have to be manually specified if *any* of the following conditions are true:

1) a top level list has more than 9 items;
2) a relevant \labelenumX or \theenumX has been redefined;
3) \item[X] has been used to manually specify labels;
4) the labels are using a font that is not the normal size and style;
5) the enumerated list is nested (i.e., not at the top level) and is therefore not using Arabic digits as labels.

For example:

```
\begin{enumerate}[\IEEEsetlabelwidth{12)}]
\item blah
\item blah
.
.
% 12 items total
\end{enumerate}
```

11.3 Description

Generally speaking, the longest label width will always have to be specified for description lists. Furthermore, the author may wish to use `\IEEEmathlabelsep` for `\labelsep` when building a math symbol list. For example:

```
\begin{description}[\IEEEsetlabelwidth{$\alpha\omega
\pi\theta\mu$}\IEEEusemathlabelsep]
\item[$\gamma\delta\beta$] Is the index of..
\item[$\alpha\omega\pi\theta\mu$] Gives the..
.
.
\end{description}
```

Sometimes it can be difficult to ascertain from inspection which of the labels is the longest. For such cases, a little diagnostic code may be helpful to measure a length and then to display the result on the console:

```
\newlength{\mydiaglen} % put in preamble
.
.
\settowidth{\mydiaglen}{$\alpha\beta\gamma$}
\showthe\mydiaglen
```

12 Theorems and Proofs

Theorems and related structures such as axioms, corollaries and lemmas, are handled in the traditional LaTeX fashion. The user must first declare the structure name via the

```
\newtheorem{struct_type}{struct_title}[in_counter]
```

command where *struct_type* is the user chosen identifier for the structure, *struct_title* is the heading that is used for the structure and *in_counter* is an optional name of a counter whose number will be displayed with the structure number and whose update will reset the structure counter. Most IEEE papers use sequential theorem numbering throughout the entire work, so an *in_counter* is usually not specified. However, those papers that do use *in_counter* usually use "section" such that the section number is the first part of each theorem number. After the structure is defined it can be used via

```
\begin{struct_type}[extra_title]
.
.
\end{struct_type}
```

where *extra_title* is an optional name that is displayed with the structure.

For example, the most common way to do theorems would be to use

```
\newtheorem{theorem}{Theorem}
```

followed as needed by environments like

```
\begin{theorem}[Einstein-Podolsky-Rosenberg]
```

Sometimes it is desirable that a structure share its counter with another structure. This can be accomplished by using the alternate form of `\newtheorem`

```
\newtheorem{struct_type}[num_like]{struct_title}
```

where *num_like* is the name of an existing structure.

IEEE theorem numbers are prefixed by the section number they were defined in (e.g., 2.5). This presents a difficulty with appendices (especially when numbered with Roman numerals) because the theorem numbers will not be unique. To remedy this, within Roman numbered appendices, IEEEtran will add an "A" prefix (e.g., A2.5). For Alpha number appendices, theorem numbering is more straightforward (e.g., A.5, B.5, etc.). For a single appendix, a constant "A" prefix is used (e.g., A.5).

12.1 Proofs

Proofs are easily handled by the predefined IEEEproof environment:

```
\begin{IEEEproof}
.
.
\end{IEEEproof}
```

The Q.E.D. symbol "□" is automatically placed at the end of each proof. If needed, the symbol can be manually accessed via the `\IEEEQED` command. Both the closed (default) "■" and open "□" forms are provided as `\IEEEQEDclosed` and `\IEEEQEDopen`, respectively. To change the default from closed to open (some journals and/or authors prefer the open form), just redefine `\IEEEQED` as desired:

```
\renewcommand{\IEEEQED}{\IEEEQEDopen}
```

IEEEproof also supports an optional argument which allows the default string "Proof" to be overridden:

```
\begin{IEEEproof}[Proof of Theorem \ref{thm:my}]
```

13 End Sections

13.1 Appendices

The `\appendix` command is used to start a single appendix. An optional argument can be used to specify a title:

```
\appendix[Proof of the Zonklar Equations]
```

After issuing `\appendix`, the `\section` command will be disabled and any attempt to use `\section` will be ignored and will cause a warning message to be generated. (The single appendix marks the end of the enumerated sections and the section counter is fixed at zero—one does not state "see Appendix A" when there is only one appendix, instead "see the Appendix" is used.) However, all lower `\subsection` commands and the `\section*` form will work as normal as these may still be needed for things like acknowledgments.

`\appendices` is used when there is more than one appendix section. `\section` is then used to declare each appendix:

```
\section{Proof of the First Zonklar Equation}
```

The mandatory argument to section can be left blank (`\section{}`) if no title is desired. It is important to remember to declare a section before any additional subsections or labels that refer to section (or subsection, etc.) numbers. As with `\appendix`, the `\section*` command and the lower `\subsection` commands will still work as usual.

There are two appendix numbering conventions used by IEEE. Capital letters (e.g., "Appendix B") and Roman numerals (e.g., "Appendix II"). The former appears to be more popular and is the IEEEtran default. Use the IEEEtran class option romanappendices to get Roman numbered appendices.

Some authors prefer to have the appendix number to be part of equation numbers for equations that appear in an appendix. This can be accomplished by redefining the equation numbers as

```
\renewcommand{\theequation}{\thesection.\arabic{equation}}
```

before the first appendix equation. For a single appendix, the constant "A" should be used in place of `\thesection`.

13.2 Acknowledgments

Acknowledgments and other unnumbered sections are created using the `\section*` command:

```
\section*{Acknowledgment}
\addcontentsline{toc}{section}{Acknowledgment}
```

The second, optional, command is needed to manually add such sections to the table of contents (which is rarely used, but some authors may do so with draft papers) as well as the document's PDF bookmarks (if using hyperref.sty).

Note that IEEE Computer Society papers typically use the plural form "Acknowledgments".

13.3 Bibliographies

Bibliographies are most easily (and correctly) generated using the IEEEtran BibTeX package [32] which is easily invoked via

```
\bibliographystyle{IEEEtran}
\bibliography{IEEEabrv,mybibfile}
```

See the IEEEtran BibTeX package documentation for more information.

When submitting the document source (.tex) file to external parties, it is strongly recommended that the BibTeX .bbl file be manually copied into the document (within the traditional LaTeX bibliography environment) so as not to depend on external files to generate the bibliography and to prevent the possibility of changes occurring therein.

13.4 Biographies

Biographies for journal articles are created using the IEEEbiography environment which supports an optional argument for the inclusion of a photo:

```
\begin{IEEEbiography}[{\includegraphics[width=1in,height=1.25in,clip,keepaspectratio]{./shell}}]{Michael Shell}
.
.
\end{IEEEbiography}
```

Note the extra set of braces that are required to prevent the LaTeX parser from becoming confused when commands with optional arguments are used within an optional argument of another command. Alternatively, a LaTeX macro (command) could be defined to facilitate a shorthand notation for the author photos. If the optional argument is not used, space will be reserved for a photo and the message "PLACE PHOTO HERE" will be displayed in place of a photo.

IEEEtran is a tad overly cautious about preventing the IEEEbiography photo area from being broken across pages. If it looks as though a IEEEbiography should be able to be "squeezed" at the end of a page, but instead it begins on a new page, try inserting

```
\vspace*{-2\baselineskip}
```

or so before the IEEEbiography and see if it can fit.

IEEE's algorithm for spacing around biographies can be a tad complex because esthetics must be considered. IEEEtran places `\vfil` above biographies. This allows the user to shove biographies down or up as desired by placing the infinitely more stretchable `\vfill` before or after the biographies.

The photo area is 1 in wide and 1.25 in long. The IEEE recommends that author photo images should be of 220 dpi (dots per inch) resolution and in gray scale with 8 bits/sample.

If no photo is available, the `\IEEEbiographynophoto` environment, which does not support an optional argument or reserve space for a photo, can be used instead.

14 Last Page Column Equalization

The IEEE (coarsely) equalizes the lengths of the columns on the last page. The balance is coarse in the sense that reference or IEEEbiography entries are not usually broken—so the column lengths are not usually perfectly equal.

Balancing the last two columns is especially important for camera ready work. It is recommended that authors use the manual approach by putting in `\newpage` at the appropriate point or `\enlargethispage{-X.Yin}` somewhere at the top of the first column of the last page where "X.Yin" is the amount to effectively shorten the text height of the given page.

Sometimes such a command has to be located between bibliography entries. This can be a problem because, although the command can be placed within the .bbl file, it will get overwritten the next time BibTeX is run. For this situation, IEEEtran offers a way to invoke commands just before a given reference number via the `\IEEEtriggeratref{}` command. For instance, issuing the command

```
\IEEEtriggeratref{10}
```

before the bibliography will insert a page break just before reference number ten. The command that is executed defaults to `\newpage`. However, this can be changed via the `\IEEEtriggercmd` command:

```
\IEEEtriggercmd{\enlargethispage{-5.35in}}
```

Note that manually set break points or page sizes will have to be readjusted if the document content ever changes.

There are LaTeX packages, such as balance.sty [33] and flushend.sty [34], that are designed to automatically balance the columns on the last page. Flushend does not require the placement of any special command in the first column of the last page, balance.sty may. However, the use of these packages is not recommended because they are known to be less than perfectly reliable in their operation. The author of balance.sty does not guarantee that it will work with every possible type of page, especially pages with figures. Under certain circumstances, flushend.sty will cause a spacing anomaly between two lines within a reference in the second column of the last page (becomes larger than the space between references). This problem seems to result

because the bibliography in IEEEtran is a list with zero space between the list items which are in footnotesize. The problem can also occur under article.cls for the same type of list. It may be possible to manually correct the flushend anomaly by tweaking the spacer at the column break via a flushend command such as "`\atColsBreak{\vskip-2pt}`", but having to do so partially defeats the purpose of using the package in the first place. If using flushend.sty or balance.sty, be sure to check the document carefully for any spacing problems—especially on the last page.

APPENDIX A
INSTALLING IEEETRAN

First of all, users should be aware that, depending on the target operating system of the IEEEtran archive packaging (e.g., .tar.gz for Unix, or .zip for MS Windows), the plain text based IEEEtran files (.bst, .cls, .sty, .tex, etc.) may use one of two different types of end-of-line character conventions. Unix (including Mac OS X) systems use line feed `<lf>` (`0x0A`), while MS Windows systems use carriage return/line feed pairs `<cr><lf>` (`0x0D 0x0A`) to signal the end of lines.[10] Most modern LaTeX systems are tolerant of differing end-of-line conventions, but some text editors aren't. (Symptoms here include text appearing all on one long line, double spacing, etc.)

LaTeX .cls files can be accessed system-wide when they are placed in the `<texmf>/tex/latex` directory, where `<texmf>` is the root directory of the user's TeX installation. On systems that have a local texmf tree (`<texmflocal>`), which may be named "`texmf-local`" or "`localtexmf`", it may be advisable to install packages in `<texmflocal>`, rather than `<texmf>` as the contents of the former, unlike that of the latter, are preserved after the LaTeX system is reinstalled and/or upgraded.

It is recommended that the user create a subdirectory `<texmf` or `texmflocal>/tex/latex/IEEE` for all IEEE related LaTeX class and package files. On some LaTeX systems, the directory look-up tables will need to be refreshed after making additions or deletions to the system files. For TeX Live systems this is accomplished via executing

```
texhash
```

as root. MiKTeX users can run

```
initexmf -u
```

to accomplish the same thing.

Users not willing or able to install the files system-wide can install them in their personal directories, but will then have to provide the path (full or relative) in addition to the filename when referring to them in LaTeX.

APPENDIX B
POSTSCRIPT/PDF OUTPUT

Some LaTeX systems are not properly configured to produce quality PostScript and/or PDF output. This has historically been more of a problem with IEEE-related work because the unique font combination the IEEE uses has been known to trigger problems with some LaTeX setups. Fortunately, these types of problems are now relatively uncommon on modern LaTeX systems.

To assist IEEE authors in detecting and correcting problems with LaTeX PostScript/PDF generation, the "Testflow" diagnostic suite was developed [35]. Authors are encouraged to take the time to go through the testflow diagnostic and identify and correct potential problems *before* their LaTeX systems have to be relied on for production work. Papers with problems such as incorrect margins, font types, PDF format errors and/or improper font embedding can incur delays during the manuscript acceptance process.

APPENDIX C
OTHER USEFUL OR RELATED EXTERNAL PACKAGES

C.1 The acronym.sty Package

Tobias Oetiker's acronym.sty [36] may be useful with papers that have a lot of acronyms. However, beware of a compatibility issue between the acronym environment and the IEEEtran description lists (see Appendix E).

C.2 The url.sty Package

Papers that contain URLs, email address, etc., can likely benefit from the use of Donald Arseneau's url.sty LaTeX package [37] which provides for more intelligent line breaking within such structures. Note that IEEEtran.cls automatically sets the url font style of url.sty to "same" (that is, URLs will be rendered in the same font as the text they appear in) as IEEE journals do. To override this, the author must place the `\urlstyle` after `\begin{document}`.

C.3 The IEEEtrantools Package

Some of the unique commands provided by the IEEEtran LaTeX class may be of use in non-IEEE related work using other class files (e.g., dissertations, technical reports, etc.). The IEEEtrantools.sty package [38] provides several popular IEEEtran commands including `\IEEEPARstart`, the IEEE style IED list environments, the IEEEeqnarray family of commands, the IEEEproof environment and `\IEEEauthorrefmark`. The IEEEtrantools package is not needed under, and should not be loaded with, the IEEEtran class. See the IEEEtrantools documentation for more details.

APPENDIX D
COMMON USER MISTAKES

Many user mistakes with IEEEtran involve doing too much rather than too little. Older class files may have required hacks in order to get the formatting closer to that of the IEEE. These tweaks are no longer needed. Users should carefully check all the loaded packages to ensure that they are still useful under the latest version of IEEEtran. Don't load packages just because "this is the way it always has been done." The same is true for manually adjusted spacing, margins, paper sizes, etc.

Below are a few of the more commonly encountered mistakes to avoid.

10. The fact that different conventions exist for plain text is, of course, an absurdity in itself. See the Wikipedia article "Newline" at http://en.wikipedia.org/wiki/Newline for the history and details.

Placing labels before captions: This is considered to be one of the most frequent mistakes made in LaTeX of all time. Remember that `\label` must be placed after or within `\caption` to be able to reference figures/tables properly. As it is `\caption` that actually sets up the reference counter, `\label`'s placed prior to `\caption` will refer to the section number, instead of the desired figure/table number.

Altering the default fonts: Authors should allow IEEEtran to manage the fonts. Unless specifically instructed otherwise, such as under comsoc mode or in the author instructions of the specific conference/journal being submitted to, do not attempt to use packages that override the default fonts such as pslatex, mathptm, etc.

Altering the default spacings, section heading styles, margins or column style: Authors should not attempt to manually alter the margins, paper size (except as provided in IEEEtran class options) or use packages that do so (geometry.sty, etc.). There should be no need to add spacing around figures, equations, etc., (except possibly for double column floats as described in Section 10.4).

Using bitmapped graphics for line art: LaTeX has always favored the use of Encapsulated PostScript (EPS) or under pdfLaTeX, Portable Document Format (PDF), (which can be considered to be a type of subset of PostScript), for graphics (see Section 10.1 for more information), and for good reason. EPS/PDF supports both vector (that is, containing objects such as lines, circles, etc., that are mathematically described) and bitmap (that is, containing only samples in the form of pixels) images. The former should always be used for drawings, graphs, charts, etc., while the latter usually has to be employed with photos (because their contents usually cannot be easily described mathematically). The drawing and graphing tools used by the author should be capable of outputting *directly*[11] in vector (EPS or PDF) format. Vector EPS/PDF images can be scaled, rotated and magnified without undergoing degradation such as pixelization or becoming gray or "jaggedy." For photos, the IEEE recommends the use of EPS/PDF (which is easy to directly import into (pdf)LaTeX in a portable manner), PNG or TIFF. For author photos JPEG (JPG) is usually acceptable. The use of other graphic formats such as BMP, EMF, VSD, etc., is unacceptable for IEEE journals. Some IEEE conferences may be more liberal with regard to the types of graphics formats they accept.

Using bitmapped fonts and/or not embedding and subsetting all document fonts: Authors should check their system with the testflow diagnostic [35] to ensure that only vector (Type 1) fonts are being used and that all fonts are embedded and subsetted. A document that uses bitmapped fonts and/or fails to contain all (and only) the needed font glyphs may be rejected by the IEEE. Watch out for graphical drawing applications that produce output with these problems (suspect this if the problem goes away when the figures are not included).

Using older graphics packages: Authors should not use anything other than the graphics and/or graphicx (preferred) package for figures. Older interfaces such as psfig, epsf, etc., have been obsolete for many years.

11. Once an image in EPS/PDF vector form is converted to a bitmap form (GIF, PNG, TIFF, JPEG, etc.) it will almost always be irretrievably locked into bitmap form even if it is later converted back into EPS/PDF.

Failing to properly divide long equations: It is the author's responsibility to ensure that all equations fit within the width of their columns. Admittedly, breaking an equation is not always easy to do and two column formatting places serious constraints on allowed equation width. However, only the author can divide his/her equation without unintentionally altering its meaning or affecting readability. Using subfunctions is a valid way to reduce to width of an equation, but altering the math font size is not.

Manually formatting references: Not only is this error prone, but requires a lot of work as well. It is better to use the IEEEtran BibTeX style [32].

Appendix E
Known Issues

acronym.sty: The acronym environment will have a problem with IEEEtran because of the modified IEEE style description list environment. The optional argument of the acronym environment cannot be used to set the width of the longest label. A workaround is to use `\IEEEiedlistdecl` to accomplish the same thing:

```
\renewcommand{\IEEEiedlistdecl}{\IEEEsetlabelwidth{SONET}}
\begin{acronym}
.
.
\end{acronym}
\renewcommand{\IEEEiedlistdecl}{\relax}% reset back
```

cite.sty: Versions prior to 5.0 (2009-03-20) will not hyperlink citation numbers under hyperref.sty.

hyperref.sty: Versions prior to 6.72u will interfere with the optional argument to `\appendix`.

Small caps font variations: The small caps font used in the free LaTeX systems have about 80% the height of normal sized letters. However, the small caps font the IEEE uses in the journals is slightly smaller with a ratio of around 75%. So, the widths of the section headings produced under the free LaTeX systems will be slightly wider than that used in actual journals. The small caps font used in many commercial LaTeX systems (such as those from YandY) has a ratio of about 65%. So, those systems will produce section headings that are narrower than those in IEEE publications. Such variations should not be cause for concern.

Appendix F
The IEEEeqnarray Commands

(*Optional—for advanced users*)

Virtually all LaTeX alignment commands such as `\eqnarray`, `\array` and `\tabular` are based on the TeX command `\halign`. LaTeX's goal of simplifying the use of `\halign` is noble. However, in hiding much of the lower level interface, a fair degree of flexibility is lost. This has resulted in the development of several packages such as amsmath [11], array.sty [28], and the MDW tools [17], each of which provides much more powerful alignment structures.

IEEEtran also provides its own unique set of alignment tools which are known as the IEEEeqnarray family. The design philosophy of the IEEEeqnarray family is to provide

a LaTeX alignment interface that is based more closely on the underlying `\halign`, but to couple this with high level column definition management and automated preamble building mechanisms (which are tedious to do in TeX). As a result, the IEEEeqnarray family of commands are flexible enough to be almost universal replacements for all the other LaTeX commands for producing multiline equations and aligned box structures such as matrices and tables of text and/or mathematics. Because the user is shielded less from the `\halign` underpinnings, the rules of operation are more involved. So, the IEEEeqnarray commands are aimed primarily toward the more advanced LaTeX users.

The use of the IEEEeqnarray family of tools described in this section is totally *optional*. The IEEEeqnarray code is self-contained and does not depend on other alignment packages—which can be used along side, or in place of, it. The IEEEtrantools.sty package (See Appendix C.3) is available for those who wish to use the IEEEeqnarray family outside of IEEEtran.cls.

Recommended sources of information on the use of IEEEeqnarray include Stefan M. Moser's *How to Typeset Equations in LaTeX* [6] and Tobias Oetiker's *The Not So Short Introduction to LaTeX 2ε* [5].

F.1 IEEEeqnarray

The IEEEeqnarray environment is for multiline equations that occupy the entire column. It is used in much the same way as `\eqnarray`, but with two additional arguments, one of which is mandatory and the other is optional:

```
\begin{IEEEeqnarray}[decl]{cols}
.
\end{IEEEeqnarray}
```

The optional argument is for commands that are to be executed within the environment, but before the alignment actually begins. This is just like the local control of the IEEEtran IED list environments. There is also a global control, `\IEEEeqnarraydecl`, which is executed just prior to the local control. By default, `\IEEEeqnarraydecl` is defined to be `\relax`. As mentioned in Section 11, users should be careful not to allow unwanted spaces to occur in these controls because such things will appear just before the IEEEeqnarray structure. Furthermore, remember that, to prevent the LaTeX parser from becoming confused, the contents of an optional argument must be enclosed in braces if the argument contains commands with optional arguments.

The mandatory argument *cols* contains the column and inter-column separator spacing ("inter-column tabskip glue" in TeXspeak) type specifiers. Column types are identified by letters. Several predefined column types are available as shown in Table 4. There are two kinds of glue types. Predefined glue types are indicated by various punctuation marks as shown in Table 5. User defined glue types are indicated by numbers.

The rules for placing these specifiers are as follows: (1) no two glue specifiers can appear next to each other—they are not additive and must be separated from each other by at least one column specifier; (2) zero inter-column spacing will be assumed between back-to-back column specifiers; (3) because of rule one, back-to-back numerals will be considered as being a single glue specified by the numerical value represented by all the digits; (4) a multiletter column specifier can be accessed by enclosing the letters within braces (otherwise it would be interpreted as being several single letter column specifiers). Because of rule three, braces are not needed around multidigit glue specifiers; (5) there must be at least one column specifier, but there is no fixed upper limit of how many columns can be supported; and (6) for `\IEEEeqnarray`, "+" centering glue will be assumed at each end of the *cols* specification if no column glue is specified there. This results in a centered structure like `\eqnarray` (the 1000pt minus 1000pt glue on each side "compresses" as needed from each side of the main text column to center that which is between). Also, `\IEEEeqnarray` automatically adds a hidden column for equation numbers to the right of the last specified column. Currently, there is no support for equation numbers on the left side.[12]

TABLE 4
IEEEeqnarray Predefined Column Types

I.D.	Description	I.D.	Description
l	left math	v	vertical rule
c	centered math	vv	two vertical rules
r	right math	V	double vertical rule
L	left math with ords	VV	two double vertical rules
C	centered math with ords	h	horizontal rule
R	right math with ords	H	double horizontal rule
s	left text	x	empty
t	centered text	X	empty math
u	right text		

Note: S, T, U, p, and P are likely to be used in future versions.

TABLE 5
IEEEeqnarray Predefined Column Separation (Glue) Types

I.D.	Width*	I.D.	Width
!	−1/6 em	.	0.5\arraycolsep
,	1/6 em	/	1.0\arraycolsep
:	2/9 em	?	2.0\arraycolsep
;	5/18 em	*	0pt plus 1fil
'	1 em	+	1000pt minus 1000pt
"	2 em	-	0pt

*All em values are referenced to the math font.
1 em = `\quad`, 2 em = `\qquad`

F.2 Defining Column Types

New column types are defined with the

```
\IEEEeqnarraydefcol{col_id}{predef}{postdef}
```

command. The *col_id* argument contains the name of the column specifier which should consist only of one or more letters. A given column specifier, even the predefined ones, can be redefined at will without warning or error.[13] The *predef* argument contains the commands that will be inserted before each cell in the column. The *postdef*

12. This is not to say that its impossible with the existing capability, just ugly.

13. Thus allowing new predefined column types to be added without breaking existing code.

argument contains the commands that will be inserted after each cell in the column. For example,

```
\IEEEeqnarraydefcol{g}{\hfil$\clubsuit$}{$\diamondsu
it$\hfil}
```

Will define a "g" text column which will place club and diamond suit symbols on either side of a cell's contents and center the respective structure within the cell. e.g.,

♣Hello♢

Using `\hfil` to control cell alignment allows the user to override the column alignment on a cell-by-cell basis by placing the infinitely more stretchable `\hfill` on one or both sides of a cell's contents. `\hfill` can even be placed between items in a cell to force them apart and against the "cell walls." The IEEEeqnarray predefined columns are designed to allow user overrides via `\hfill` whenever possible (even for the math mode cells).

Please note that TEX will not allow unmatched braces within the arguments of commands. If braces are needed, such as for the argument of a command, they will have to be provided within the cells themselves. For example,

```
\IEEEeqnarraydefcol{myp}{\parbox[c]{0.5in}}{}
\begin{IEEEeqnarraybox}{{myp}c}
{first\\second}&\alpha\\
&\beta%
\end{IEEEeqnarraybox}
```

defines a column type named "myp" that will place text within a 0.5 inch wide parbox which is centered on the cell's baseline. Note that because the column type name consists of more than one letter, it has to be enclosed within an extra set of braces in the column specifications or else it would be interpreted as three adjacent columns "m", "y" and "p". Also, the contents of the cell must be enclosed within braces so that (1) the `\parbox` command sees the entire contents as its argument; and (2) the newline within the parbox will not be interpreted as being the end of the alignment row. Be aware that it can happen that a column is given an empty cell, such as in the second row in the example, or when entering blank separator rows. When this happens, a `\relax` will appear in the column which will be acquired as the command's argument. Therefore, commands in column definitions that acquire arguments from the cells should not choke if fed `\relax`.

For reference, the definitions of the predefined column types are shown here:

```
% math
\IEEEeqnarraydefcol{l}{$\IEEEeqnarraymathstyle}{$\hfil}
\IEEEeqnarraydefcol{c}{\hfil$\IEEEeqnarraymathstyle}{$\hfil}
\IEEEeqnarraydefcol{r}{\hfil$\IEEEeqnarraymathstyle}{$}
\IEEEeqnarraydefcol{L}{$\IEEEeqnarraymathstyle{}}{{}$\hfil}
\IEEEeqnarraydefcol{C}{\hfil$\IEEEeqnarraymathstyle{}}{{}$\h
fil}
\IEEEeqnarraydefcol{R}{\hfil$\IEEEeqnarraymathstyle{}}{{}$}
% text
\IEEEeqnarraydefcol{s}{\IEEEeqnarraytextstyle}{\hfil}
\IEEEeqnarraydefcol{t}{\hfil\IEEEeqnarraytextstyle}{\hfil}
\IEEEeqnarraydefcol{u}{\hfil\IEEEeqnarraytextstyle}{}
% vertical rules
\IEEEeqnarraydefcol{v}{}{\vrule width\arrayrulewidth}
\IEEEeqnarraydefcol{vv}{\vrule width\arrayrulewidth\hfil}{\h
fil\vrule width\arrayrulewidth}
\IEEEeqnarraydefcol{V}{}{\vrule width\arrayrulewidth\hskip\d
oublerulesep\vrule width\arrayrulewidth}
\IEEEeqnarraydefcol{VV}{\vrule width\arrayrulewidth\hskip\do
ublerulesep\vrule width\arrayrulewidth\hfil}%
{\hfil\vrule width\arrayrulewidth\hskip\doublerulesep\vrule
 width\arrayrulewidth}
% horizontal rules
\IEEEeqnarraydefcol{h}{}{\leaders\hrule height\arrayrulewidt
h\hfil}
\IEEEeqnarraydefcol{H}{}{\leaders\vbox{\hrule width\arrayrul
ewidth\vskip\doublerulesep\hrule width\arrayrulewidth}\hfil}
% plain
\IEEEeqnarraydefcol{x}{}{}
\IEEEeqnarraydefcol{X}{$}{$}
```

Note the inclusion of the commands `\IEEEeqnarraymathstyle` and `\IEEEeqnarraytextstyle` in the math and text columns, respectively. These commands allow the user to control the style of all of the math and text columns. However, because the changes apply to all the columns, the user will have to define new column types if different styles are needed in the same alignment (or different styles can be manually specified in each cell). The default definitions for these commands are

```
\newcommand{\IEEEeqnarraymathstyle}{\displaystyle}
\newcommand{\IEEEeqnarraytextstyle}{\relax}
```

which allows the text columns to be in whatever style was in effect when the alignment was started and the default math style will be in display style, but can be easily changed as needed. e.g.,

```
\begin{IEEEeqnarray}[\renewcommand{\IEEEeqnarraymath
style}{\scriptstyle}]{rCl}
```

will result in script style math columns.

The columns relating to vertical and horizontal lines will be discussed in Appendix F.11 as they are typically used only when producing tables.

The "x" and "X" columns are basic empty text and math mode columns without any formatting or style controls.

F.3 Defining Glue Types

New column separation glue types are defined with the

```
\IEEEeqnarraydefcolsep{colsep_id}{def}
```

command. The *colsep_id* argument contains the number of the column separation glue specifier which should consist only of numerals. Different glue type names must have different numerical values. (Don't get too cute—"007" is identical to "7".) User defined column glue specifiers can be redefined at will without warning or error. The *def* argument contains the width of the given glue type. Widths may be specified as absolute values or reference length commands:

```
\IEEEeqnarraydefcolsep{9}{10pt}
\IEEEeqnarraydefcolsep{11}{2\tabcolsep}
```

The glue type widths are *not* evaluated when defined, but are evaluated each time they are actually referenced as IEEEeqnarray column specifiers. Thus, for the second definition in the example above, if `\tabcolsep` were to be revised after the glue type was defined, the revised value would be what is used.

Rubber lengths are allowed too. The fact that the centering glue "+" is a known value can be exploited to achieve some interesting effects. For example,

```
\IEEEeqnarraydefcolsep{17}{200pt minus 200pt}
```

will produce a column separation glue that is always $1/5$ of the width of the distance from the equation sides to the ends of the main text columns. And, of course, "+" can be used

as needed to produce groups of equally spaced equations as in amsmath's \align:

```
\begin{IEEEeqnarray}{Rl+Rl+Rl}
```

F.4 A Simple Example of Use

The example in Section 9 can be implemented using \IEEEeqnarray via

```
\begin{IEEEeqnarray}{rCl}
Z&=&x_1 + x_2 + x_3 + x_4 + x_5 + x_6\IEEEnonumber\\
&&+\:a + b%
\end{IEEEeqnarray}
```

As shown in Table 4 the "C" column type is a centered math mode column with empty ords ("{}") on either side. So, there is no need to place empty ords around the equal sign. As with \eqnarray, the & separate the column cells and are where the inter-column separation glue will appear (when nonzero).

Note the presence of the % at the end of the second row. TEX does *not* ignore spaces that occur *before* commands or & column delimiters, but does ignore those that occur *after*. Most LATEX alignment implementations shield the user from this behavior by removing all spacing previous to &, \\ and \end. The IEEEeqnarray family does not do this! So, it is important to prevent spaces (including implied ones at the end of lines) from occurring before such commands unless they are wanted. Suspect this problem if there is an unexplained offset within a column. In the given example, unwanted spacing is not an issue because end spacing is ignored in math mode anyway. However, it would be an issue if the columns used text mode instead.

Alternatively, one could use a two column form:

```
\begin{IEEEeqnarray}{Rl}
Z=&x_1 + x_2 + x_3 + x_4 + x_5 + x_6\IEEEnonumber\\
&+\:a + b%
\end{IEEEeqnarray}
```

F.5 Equation Numbering

Like \eqnarray, \IEEEeqnarray, has a "star form," \IEEEeqnarray*, which does not place equation numbers at the end of each row by default. The default behavior of individual rows can be overridden by placing the commands \IEEEyesnumber or \IEEEnonumber as needed in the last column. \IEEEeqnarray also provides \IEEEyessubnumber and \IEEEnosubnumber which can be used to enable or disable a subequation number for the given row. To support this feature, IEEEtran defines its own IEEEsubequation counter (reset with changes to equation) and \theIEEEsubequation command.[14]

As of version 1.8 of IEEEtran, the star forms \IEEEyesnumber*, \IEEEnonumber*, \IEEEyessubnumber* and \IEEEnosubnumber* are available which persist across rows until countermanded by another star command. The behavior of later individual rows can be selectively overridden with the use of the non-star forms as needed.

Despite there being four numbering commands, it is useful to remember that there are only three IEEEeqnarray numbering modes:

1) Display nothing and do not alter the counters (\IEEEnonumber);
2) Increment the equation counter and display an equation number without a subequation part (\IEEEyesnumber);
3) Increment the subequation counter and display an equation number with a subequation number (\IEEEyessubnumber).

\IEEEnosubnumber is not really needed and behaves much like \IEEEyesnumber except that the former does not also enable equation numbering if it isn't already on (and does not alter the numbering properties of the current row if equation numbering is off).

Generally speaking, only one numbering command should be used per row. In particular, mixing yes and no commands on a single row could result in unintended operation. However, a notable exception is the very useful \IEEEyesnumber\IEEEyessubnumber combination which starts a new subequation sequence. For example,

```
\begin{IEEEeqnarray}{c}
x1\IEEEyesnumber\IEEEyessubnumber*\\
x2\\
x3\IEEEyesnumber\IEEEyessubnumber\label{eqn:expl}\\
x4\\
x5\IEEEyesnumber*\\
x6
\end{IEEEeqnarray}
```

yields:

$$x1 \tag{8a}$$
$$x2 \tag{8b}$$
$$x3 \tag{9a}$$
$$x4 \tag{9b}$$
$$x5 \tag{10}$$
$$x6 \tag{11}$$

The \IEEEyesnumber command increments the equation counter of what would otherwise have been a subequation row, resets the subequation counter and turns off subequation numbering. The following \IEEEyessubnumber then increments the subequation counter by one and restores subequation numbering.[15]

Note that any labels for (sub)equations must be placed *after* any numbering control command(s) because, prior to that point, the label will reference the equation number *that would have been used* if there had not been any numbering control commands.

Please be aware that \IEEEeqnarray, like \eqnarray, will, if the equation is long enough, overwrite the equation number without warning![16] For cases when this happens, users can insert a \IEEEeqnarraynumspace command at

14. What is actually displayed is the \theIEEEsubequationdis command.

15. Invoking only a \IEEEyessubnumber following a normal equation number line will result in a sequence like 14, 14a. The IEEE does not typically use normal equation numbers followed by subequations carrying that same base equation number, but the capability is there if you ever need it. IEEEtran versions prior to v1.8 differed here in that they automatically advanced the equation number on the "first" call to \IEEEyessubnumber and so did not have this degree of flexibility.

16. This behavior is extremely difficult to avoid if the equation line is to remain centered irrespective of the width of the equation number—without even considering the subjective question of how close is too close for any given case!

the end of the line (after any \IEEEyessubnumber if used) which will insert a space equal in width to the displayed equation number:

```
··· + x_z \IEEEyessubnumber\IEEEeqnarraynumspace\\
```

As a result, the entire multiline equation will be slightly shifted to the left. The IEEE often does the same thing in its journals when confronted by this situation. If an overfull hbox results, the offending equation line will have to be further divided.

F.6 Extra Vertical Spacing and Page Breaks

Like \eqnarray, \IEEEeqnarray's \\ command supports a star form which inhibits page breaks at the given line as well as an optional extra vertical spacing argument:

```
&+\:a + b\\*[5pt]
```

Users are reminded from Section 9 that amsmath will configure LaTeX to disallow page breaks within multiline equations—including those made by \IEEEeqnarray because it also honors the value of \interdisplaylinepenalty.

Also like \eqnarray, \IEEEeqnarray normally places a small amount of extra spacing (as specified by the length command \jot) between lines to "open up" equations as well as to prevent large symbols from coming to close to the lines above them.

F.7 IEEEeqnarraybox

\IEEEeqnarray is not suitable for producing structures such as matrices and tables because it must have exclusive access to the main text column and cannot be nested within other structures. For these applications, the \IEEEeqnarraybox command is provided. \IEEEeqnarraybox differs from \IEEEeqnarray in the following ways:

1) the entire contents are wrapped within a box and therefore, can be nested within other display or alignment structures (such as \equation, \IEEEeqnarray or even another \IEEEeqnarraybox). Note that, like all box structures, page breaks are *not* allowed between the rows of an \IEEEeqnarraybox;
2) the default glue at the outer ends of the first and last columns is 0 pt ("-"), not "+" centering glue as with \IEEEeqnarray;
3) no automatic (hidden) equation number column is provided;
4) the star form "\IEEEeqnarraybox*" turns off the extra \jot vertical spacing after each row;
5) \IEEEeqnarrayboxdecl is the global control.

There are two subforms: \IEEEeqnarrayboxm which is for use within math modes and is analogous to \array; and \IEEEeqnarrayboxt which is for use within text modes and is analogous to \tabular. If called via \IEEEeqnarraybox the current math/text mode will be auto-detected and the proper subform will automatically be selected. Therefore, \IEEEeqnarraybox can replace \array as well as \tabular.

The syntax for \IEEEeqnarraybox is similar to \IEEEeqnarray, but with two additional optional arguments:

```
\begin{IEEEeqnarraybox}[decl][pos][width]{cols}
.
\end{IEEEeqnarraybox}
```

The *pos* argument can be one of t, c, or b to control where the box will be vertically aligned relative to the current baseline: t at the top row; c at the center;[17] b at the bottom row. The default is b.

The *width* argument specifies the width the box. **Warning:** if a width is specified, there *must* be one or more rubber lengths in the inter-column glue specifiers (such as that of "*" or "+") so that the box can be sized as needed. If there is no such glue, or the glue provided cannot stretch/shrink enough as needed, the box cannot be sized to *width* and an underfull or overfull hbox error will result. If no *width* argument is provided, the box will be set to its natural width (and the use of rubber inter-column glue will not be required).

\IEEEeqnarraybox uses the same column and glue type specifiers/definitions as \IEEEeqnarray.

F.8 Line Spacing in LaTeX

Before discussing some of the more advanced aspects of vertical spacing control in the IEEEeqnarray family, it is important to review the details of LaTeX's line spacing algorithm. Normally, baselines are separated by the amount given by the length command \baselineskip. With each change in font size, \baselineskip is reset to the default value for that font size (multiplied by \baselinestretch). Then the value of \baselineskip is saved to the length variable \normalbaselineskip (so that the normal value can be later referenced even if \baselineskip is set to another value by the user). However, if the top of a line ever gets closer than \lineskiplimit to the bottom of the line above it, the use of \baselineskip will be suspended and \lineskip spacing will be placed between the two lines.[18]

This system works well for text. However, for mathematics, whose symbols have a much higher dynamic range of heights and depths, it is usually better to go ahead and always add an extra fixed amount of space (\jot) as mentioned in Appendix F.6.

When the IEEEeqnarray family is loaded, a new length command is defined, \IEEEnormaljot, which stores the nominal value of \jot[19], so that this can be always be referred to even if other values are currently being used.

At the start of \IEEEeqnarray/box, but before the local or global controls, the following initialization takes place:

```
\lineskip=0pt
\lineskiplimit=0pt
\baselineskip=\normalbaselineskip
\jot=\IEEEnormaljot
```

17. Centering is actually done along the "math axis" (not exactly on the text baseline, but quite close to it). Many LaTeX users are not aware of this minor distinction.

18. Within IEEEtran.cls, \lineskiplimit and \lineskip are zero—if things get too close it is the author's responsibility to correct the problem without having IEEEtran.cls second guessing the author's intent.

19. Within IEEEtran.cls, the nominal value of \jot is 25% of the baselineskip for the normalsize font.

Thus, \baselineskip is set to the normal value for the current font, \jot is restored to its nominal value and the \lineskiplimit system is disabled.[20]

This system is designed to better facilitate nested IEEEeqnarraybox structures as well as to help prevent the user from encountering seemingly uncontrollable spacing behavior (e.g., "How *do* I get rid of that unwanted space?!").

F.9 The IEEEeqnarray Strut System

When creating tables, especially tables with vertical rules, vertical space between the rows of the table is not generally desirable because such space will suspend the column cell definitions and "cut across" any vertical rules that may be present. Yet, there must be a way to keep rows adequately spaced apart. To solve this problem, the IEEEeqnarray/box commands provide an integrated system to manage struts[21] contained in a hidden column on the right end of each IEEEeqnarray/box structure.

The struts in each row will be set to a default strut height and depth. Normally, the default strut height and depth are initialized to zero, so the struts will effectively not be present. The user can set the default strut values via the

```
\IEEEeqnarraystrutsize{height}{depth}[decl]
```

command which can be placed in a local or global control. The optional argument is for commands that will be executed prior to the evaluation of the height and depth arguments. Thus,

```
\IEEEeqnarraystrutsize{0.5\baselineskip}{}[\large]
```

will set the default strut height to half the baselineskip used by the large font size, even if the current baselineskip (and/or font size) is different. The commands which are executed within the optional argument are contained within their own environment so as not to have any effects outside of the \IEEEeqnarraystrutsize command. For mimicking the action of \baselineskip, the typically recommended height and depth of struts is 70% and 30%, respectively, of \normalbaselineskip[22]. \IEEEeqnarraystrutsize will assume these values if its height and/or depth arguments are left blank. e.g., in the previous example, the strut *depth* will be set to 30% of \normalbaselineskip for the large font size.

There is also a

```
\IEEEeqnarraystrutsizeadd{height}{depth}[decl]
```

command which will *add* to the current default strut values and can be used much like the \extrarowheight parameter of the array.sty package. Empty arguments are assumed to be 0 pt.

\IEEEeqnarraystrutsize and \IEEEeqnarraystrutsizeadd can also be used at the *end of the last* column to alter the strut size used for a particular row (the default strut values of the other rows will not be affected).

There is also a

```
\IEEEstrut[height][depth][decl]
```

which produces a strut. It can be used whenever a "manual" strut is needed—even outside the \IEEEeqnarray/box environments. If a height or depth argument is not provided (or empty) then these will be assumed in the same way as \IEEEeqnarraystrutsize does.

For diagnostic purposes (in order to see if any row objects exceed the height of the struts), the command \IEEEvisiblestrutstrue can be placed with an \IEEEeqnarray/box or \IEEEstrut control to make the struts visible.

When using \IEEEeqnarraybox to produce tables that contain vertical lines, it is usually desirable to shutdown the \baselineskip system and switch over to pure strut spacing. The following command sequence, placed within a local or global control, will serve this purpose:

```
\IEEEeqnarraystrutsize{0.7\normalbaselineskip}{0.3\normalbaselineskip}[\relax]
\setlength{\baselineskip}{0pt}%
\setlength{\lineskip}{0pt}%
\setlength{\lineskiplimit}{0pt}%
\setlength{\jot}{0pt}%
```

Note the use of "%" to prevent the ends of the lines that end in braces from being interpreted as unwanted spaces. Because of the frequent need to call this sequence, the IEEEeqnarray family provides the \IEEEeqnarraystrutmode command which does the same thing.

F.10 Overriding Column Types

Within a row, one or more column types can be overridden by placing the command

```
\IEEEeqnarraymulticol{num_cols}{col_type}{text}
```

as the *very first* command in a cell. This command is the IEEEeqnarray equivalent of \multicolumn. The first argument is the number of columns to override (cutting through any inter-column glues as needed). The second argument is the column type specifier to use. The third argument contains the cell text. The third argument will have to be enclosed within an extra set of braces if the column type is to acquire it as an argument—as was done with the "myp" parbox column type in the example earlier (Appendix F.2).

There is also the \IEEEeqnarrayomit command which, when used as the very first command in a cell, will suspend the use of the normal column type for that cell. This is somewhat like a quicker version of \IEEEeqnarraymulticol{1}{x}{}.

Users are cautioned not to use commands like these (e.g., \multicolumn) that are designed for other alignment environments.[23]

F.11 Predefined Column Types for Rules

Several of the predefined column types produce vertical or horizontal lines. Note that, in the IEEEeqnarray family, rules are declared and treated as normal column types—they are not hidden. Although this approach may increase the number of columns the user has to keep track of, especially when creating tables, it does offer a great deal of flexibility

20. As long as rows cannot be of negative height.

21. "Struts" are vertical rules of zero width, but of finite height.

22. Note that this *not* the normalsize baselineskip, but the normal baselineskip for the current font size.

23. Those who are familiar with TEX may be interested in the fact that TEX's \omit, \span and \multispan should work in \IEEEeqnarraybox, but not in \IEEEeqnarray because of the need to track column usage with a hidden counter in the latter.

by allowing the user to override, or otherwise manipulate, any column type (including those that produce rules) at will.

All of the predefined rule column types use the `\arrayrulewidth` length to determine the line thickness and `\doublerulesep` for the spacing of double rules.

The "`v`" column type produces a vertical rule, "`vv`" produces two back-to-back vertical rules which will appear as one rule of twice the normal thickness. "`V`" produces a double vertical rule with `\doublerulesep` spacing between its two lines. "`VV`" produces two back-to-back double vertical rules which will appear to be three vertical rules, the middle one of which being twice as thick as the other two. It is possible to "spread apart" the "`vv`" and "`VV`" types by placing a spacer within their columns—thus they can be used to generate two single, or double, vertical rules whose separation distance is programmable.

The "`h`" and "`H`" types produce single and double horizontal rules, respectively. Horizontal rule types are not normally used in the column specifications, but rather with the `\IEEEeqnarraymulticol` command in order to draw a horizontal rule across one or more column(s).

Please be aware that the line commands of other alignment environments may not work properly within the IEEEeqnarray family which provides its own ways of doing these types of things. In particular, `\cline` is totally incompatible—users should use the `\IEEEeqnarraymulticol{`*num_cols*`}{h}{}` command instead. However, `\vline` and `\hline` should work—unless another LaTeX package redefines them in some incompatible way. The IEEEeqnarray family provides its own version of `\vline`:

```
\IEEEeqnarrayvrule[rule_thickness]
```

that produces a vertical rule extending from the top to bottom of a cell without overriding the column type. The optional argument is for specifying the rule thickness which defaults to `\arrayrulewidth` if no argument is provided.

The IEEEeqnarray row commands (discussed in the next section) provide some alternatives to `\hline`.

F.12 Row Commands

The IEEEeqnarray family has several commands which can be used to produce special rows that span all the columns. Unless otherwise noted, the commands described here must issued as the very first command in a given row.

To produce a spacer row that relies on the strut system, use

```
\IEEEeqnarrayseprow[height][decl]
```

The first argument specifies the height of the strut row, if left blank or empty, the default value of `0.25\normalbaselineskip` will be assumed. The second optional argument is for commands which will be executed prior to the evaluation of the first argument as is done with `\IEEEeqnarraystrutsize`. `\IEEEeqnarrayseprow` will *not* interrupt the column definitions, so it will not cut vertical lines. If column definition suspension is desired, use the cutting form which will override all the column types in the entire row:

```
\IEEEeqnarrayseprowcut[height][decl]
```

To produce a horizontal rule row, use:

```
\IEEEeqnarrayrulerow[rule_thickness]
```

which will override all the column definitions with one that produces a horizontal rule. If the optional rule thickness is not specified, the value of `\arrayrulewidth` will be used.

To produce a double rule row, use:

```
\IEEEeqnarraydblrulerow[rule_thickness][spacing]
```

which will produce a rule row, a (noncutting) separation row, followed by another rule row. If the optional rule thickness is not specified, the value of `\arrayrulewidth` will be used when producing each of the two rule rows. If the optional separation distance is not specified, the value of `\doublerulesep` will be used. There is also a cutting form:

```
\IEEEeqnarraydblrulerowcut[rule_thickness][spacing]
```

which works the same way except that the separation row will override all the column definitions. (Vertical rule columns will not appear inside the double rule row produced by this command.)

F.13 Useful Low Level TeX Commands

Although the use of lower level TeX commands is generally frowned upon in LaTeX, some of them are just too helpful to ignore.

`\phantom{}` produces an invisible box with the width, height and depth of its contents, but the contents themselves will not appear in the output. There is also the `\hphantom{}` and `\vphantom{}` forms which retain only the contents' width, or its height and depth, respectively. As an example, look carefully at the footnotes at the bottom of Table 5. This table was produced using the `\IEEEeqnarraybox` command. The footnotes are actually contained within the last two rows of the table. Note how the left sides of the footnotes line up, even though the first one has a superscript asterisk for a footnote symbol. The reason that the second row lines up is because, at its left side, it employs a horizontal phantom of the very same symbol:

```
\hphantom{\textsuperscript{*}}
```

Vertical phantoms can be used to equalize row height or spacing—such as to get matrices that fit within brackets of the same size even though one has "tall" symbols and the other not.

The opposite of `\hphantom{}` is `\rlap{}` which displays its contents, but with zero width. There is also an `\llap{}` which does the same thing, but the contained object will appear just to the left of the given point, rather than after as with `\rlap`. For example, look closely at the first "Width" column heading in Table 5. The word "Width" is centered irrespective of the asterisk. That is because the width of the asterisk was zeroed:

```
Width\rlap{\textsuperscript{*}}
```

The vertical analog of `\rlap` is `\smash{}` which reduces the apparent height and depth of its contents to zero. (LaTeX's `\raisebox{0pt}[0pt][0pt]{}` does about the same thing, and also provides an adjustable vertical offset.) `\smash` can be used when space is already reserved for an object, but that LaTeX does not "know" this and would allocate unwanted additional vertical space. One good use of smash for table objects that are to be "slipped" into a hidden row of zero height, or into a row which is to be no

higher than the "short" things, such as horizontal rules, that are in its other columns.

The TEX `\noalign{}` command can be used within IEEEeqnarray family to inject text which is outside of the alignment structure. For example,

```
\begin{IEEEeqnarray}{rCl}
A_1&=&7\IEEEyesnumber\IEEEyessubnumber\\
A_2&=&b+1\IEEEyessubnumber\\
\noalign{\noindent and\vspace{\jot}}A_3&=&d+2\IEEEye
ssubnumber%
\end{IEEEeqnarray}
```

produces

$$A_1 = 7 \tag{12a}$$

$$A_2 = b + 1 \tag{12b}$$

and

$$A_3 = d + 2 \tag{12c}$$

When employed, `\noalign` must be the *very* first command in a row—even before any `\IEEEeqnarraymulticol`, `\IEEEeqnarrayomit`, or row commands.

Be forewarned that the proper use of `\noalign` can be tricky. There are three potential issues. (1) Remember that `\noalign` will place its contents outside of the alignment. So, the line spacing controls of the IEEEeqnarray commands will not be in effect. The user may have to manually add `\baselineskip` and/or `\jot` spacing as needed (which was done in the previous example). (2) Furthermore, `\noalign` does *not* automatically place its contents within a box. However, nonaligned material *must* be placed within a *horizontal* box when within the vertical box produced by the `\IEEEeqnarraybox` command. Therefore, when using `\noalign` within a `\IEEEeqnarraybox`, be sure to wrap things up in an `\hbox{}`:[24]

```
\noalign{\hbox{and therefore}}
```

(3) Finally, there may be some issues related to how easily page breaks occur around the `\noalign` lines. This is an issue only with `\IEEEeqnarray` because page breaks cannot occur within the box produced by `\IEEEeqnarraybox`. If needed, page break desirability can be altered by manually entering `\pagebreak`, or `\nopagebreak`, etc., at the end of the `\noalign` contents.

F.14 More Practical Examples of Use

The use of the IEEEeqnarray is somewhat complicated. However, once the building blocks and core concepts are understood, the user may find that is easier to use the IEEEeqnarray family for just about every alignment situation rather than to have to remember all the interfaces and unique behaviors of many different tools.

A few "real world" examples will now be demonstrated.

F.14.1 IEEEeqnarray Cases Structures

Cases structures can be obtained using `\IEEEeqnarraybox`:

$$|x| = \begin{cases} x, & \text{for } x \geq 0 \\ -x, & \text{for } x < 0 \end{cases} \tag{13}$$

which was produced using the code:

```
\begin{equation}
\setlength{\nulldelimiterspace}{0pt}
|x|=\left\{\begin{IEEEeqnarraybox}[\relax][c]{l's}
x,&for $x \geq 0$\\
-x,&for $x < 0$%
\end{IEEEeqnarraybox}\right.
\end{equation}
```

Note the use of the large `\quad` (1 em) spacing before the conditional statements. The zeroing of `\nulldelimiterspace`, an optional step, eliminates the width of the nonvisible closing brace "`\right.`" in order to perfectly center the visible portion of the equation.[25]

Note that both branches share a common equation number. If an equation (sub)number is wanted for each branch, the preferred solution is to use the cases.sty package as discussed in Section 9.1. However, it is possible to use `\IEEEeqnarray` to build such a thing—although it takes extra work and a few tricks to do so. For example,

$$|x| = \begin{cases} x, & \text{for } x \geq 0 \\ -x, & \text{for } x < 0 \end{cases} \tag{14a}$$

(14b)

was produced using the code:

```
\begin{IEEEeqnarray}[\setlength{\nulldelimiterspace}
{0pt}]{rl's}
&x,&for $x \geq 0$\IEEEyesnumber\IEEEyessubnumber\\*
[-0.625\normalbaselineskip]
\smash{|x|=\left\{\IEEEstrut[3\jot][3\jot]\right.}&&
\nonumber\\*[-0.625\normalbaselineskip]
&-x,&for $x < 0$\IEEEyessubnumber
\end{IEEEeqnarray}
```

A hidden middle row is used to hold the left hand side of the equality. In order to prevent this row from altering the spacing between the two branches, its height must be smashed and the extra line spacing (which consists of `\baselineskip`, plus `\jot` which is normally `0.25\baselineskip` for IEEEtran.cls.) must be removed, half from above and half from below,—making it look as though the middle row never occurred. Because the large brace cannot "see" the height of the branches, it must be manually sized with a strut. The star form of the new line commands is used to prevent the possibility of a page break within the structure.

F.14.2 Matrices

Displayed matrices can easily be created with `\IEEEeqnarraybox`:

$$I = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{15}$$

The code of this example is quite simple:

```
\begin{equation}
I = \left(\begin{IEEEeqnarraybox*}[][c]{,c/c/c,}
1&0&0\\
0&1&0\\
0&0&1%
\end{IEEEeqnarraybox*}\right)
\end{equation}
```

Because the example matrix has elements of normal height, one can use the star form of `\IEEEeqnarraybox` to turn off the extra `\jot` component of the line spacing so as to make a more compact matrix. If larger symbols had been used in

24. LATEX's `\mbox` will *not* work!

25. The width of null delimiters is typically only 1.2 pt, and so can usually be safely ignored.

the matrix, the nonstar form would be the better choice. `\arraycolsep` typically serves quite well as an element column separator. A standard small math space is added to the ends of the matrix to provide a little distance between it and its enclosing parentheses.

It is instructive to show how to construct a "small" matrix[26],

$$S = \left[\begin{smallmatrix} 1/2 & 0 \\ 0 & 3/4 \end{smallmatrix}\right] \tag{16}$$

which was produced via

```
\newcommand{\mysmallarraydecl}{\renewcommand{%
\IEEEeqnarraymathstyle}{\scriptscriptstyle}%
\renewcommand{\IEEEeqnarraytextstyle}{\scriptsize}%
\renewcommand{\baselinestretch}{1.1}%
\settowidth{\normalbaselineskip}{\scriptsize
\hspace{\baselinestretch\baselineskip}}%
\setlength{\baselineskip}{\normalbaselineskip}%
\setlength{\jot}{0.25\normalbaselineskip}%
\setlength{\arraycolsep}{2pt}}
%
\begin{equation}
S=\left[\begin{IEEEeqnarraybox*}[\mysmallarraydecl]
[c]{,c/c,}
1/2&0\\
0&3/4%
\end{IEEEeqnarraybox*}\right]
\end{equation}
```

The use of a user defined command, `\mysmallarraydecl`, to contain the IEEEeqnarray setup code, demonstrates how users can easily recreate their most commonly used structures by fully exploiting the on-the-fly configurability of the IEEEeqnarray family.

This example is more complex than need be in order to demonstrate a few techniques. It would be easy enough to set `\baselineskip` to the desired value, but suppose that the matrix rows are to be spaced some multiple of the `\baselineskip` of the `\scriptsize` font. Complicating matters even more is the fact that most LaTeX class files will not allow the user to execute text font size commands within math mode—and the matrix is within an equation. So, `\scriptsize` cannot be used to directly set the `\baselineskip`.

The first step is to set the math and text columns to their desired styles. Then `\baselinestretch` is setup to be used like `\arraystretch`. The trick is to run `\scriptsize` within a `\settowidth` command which stores the `\baselineskip` of the `\scriptsize` font, multiplied by `\baselinestretch`, in `\normalbaselineskip` which is then used to set `\baselineskip`, `\jot`, etc. Finally, `\arraycolsep` is reduced to better suit the smaller font. Note the use of "%" to prevent unwanted spaces from appearing after the braces at the end of lines in `\mysmallarraydecl`.

F.14.3 Tables

Tables, especially those with lines, tend to be a little more complicated. Table 6 was made with the following code:

```
\begin{table}[!t]
\centering
\caption{Network Delay as a Function of Load}
\label{table_delay}
\begin{IEEEeqnarraybox}[\IEEEeqnarraystrutmode\IEEEe
qnarraystrutsizeadd{2pt}{0pt}]{x/r/Vx/r/v/r/x}
```

TABLE 6
Network Delay as a Function of Load

β	Average Delay	
	$\lambda_{\min}$	$\lambda_{\max}$
1	0.057	0.172
10	0.124	0.536
100	0.830	0.905*

*limited usability

```
\IEEEeqnarraydblrulerowcut\\
&&&&\IEEEeqnarraymulticol{3}{t}{Average Delay}&\\
&\hfill\raisebox{-3pt}[0pt][0pt]{$\beta$}\hfill&&\IE
EEeqnarraymulticol{5}{h}{}%
\IEEEeqnarraystrutsize{0pt}{0pt}\\
&&&&\hfill\lambda_{\mbox{min}}\hfill&&\hfill
\lambda_{\mbox{max\vphantom{i}}}\hfill&\IEEEeqnarray
strutsizeadd{0pt}{2pt}\\
\IEEEeqnarraydblrulerowcut\\
&1&&&   0.057&& 0.172&\\
&10&&&  0.124&& 0.536&\\
&100&&& 0.830&& 0.905\rlap{\textsuperscript{*}}&\\
\IEEEeqnarraydblrulerowcut\\
&\IEEEeqnarraymulticol{7}{s}{\scriptsize\textupersc
ript{*}limited usability}%
\end{IEEEeqnarraybox}
\end{table}
```

Because this table has lines, the first step is to enable strut mode line spacing. The strut height is then increased by a couple of points to provide a little more headroom above the letters.[27] This table uses cutting horizontal rules and open sides as is commonly done in IEEE publications. There are three extra "x" columns which serve as place holders. The "x" columns at each end serve as a quick way to get the horizontal rules to extend a little past the contents of the table. The middle "x" column serves as an attachment point for the horizontal rule that is below "Average Delay". Without this extra column, the left side of that horizontal rule would cut into the middle double vertical rule.[28] Notice how the "β" is smuggled in as part of the row containing the horizontal rule. β has to be smashed so that it will not add unwanted vertical spacing. Likewise, the strut for that row is disabled. Also, `\raisebox` is used instead of `\smash` so that β can be vertically lowered—otherwise it would appear on its baseline which is too high for the purpose at hand. The `\hfill` on either side of β changes the justification of that cell to centered. The "min" and "max" subscripts would not normally sit at the same level because the "i" in min is slightly higher than the letters in "max". To fix this, a `\vphantom` "i" is added to "max". Because these subscripts sit so low, the depth of that line's strut is increased a couple of points. Alternatively, one could have just smashed the "i". The asterisk next to "0.905" is reduced to zero width via `\rlap` so that it will not affect its cell's width or alignment. This example also illustrates how to integrate table footnotes into the end of a table without the help of external packages.

Strut spacing does not work so well for rows that contain tall symbols because such objects routinely exceed the height of the struts. Furthermore, increasing the strut height

26. IEEE authors should note that the use of small matrices is not recommended as the IEEE does not usually reduce font sizes in equations or alter the main text baselineskip to accommodate in-text mathematics.

27. Knuth calls this extra step a mark of quality.

28. Some may even think it would be better that way, but we want to show some tricks in these examples.

TABLE 7
Possible Ω Functions

Range	$\Omega(m)$
$x < 0$	$\Omega(m) = \sum_{i=0}^{m} K^{-i}$
$x \geq 0$	$\Omega(m) = \sqrt{m}$

is often not an option because (1) the height and depth of the tall symbols must be measured or guessed; and (2) there may be other rows which have normal line height. Table 7 illustrates such a situation. Its code is shown here:

```
\begin{table}[!t]
\centering
\caption{Possible $\Omega$ Functions}
\label{table_omega}
\begin{IEEEeqnarraybox}[\IEEEeqnarraystrutmode\IEEEe
qnarraystrutsizeadd{2pt}{1pt}]{v/c/v/c/v}
\IEEEeqnarrayrulerow\\
&\mbox{Range}&&\Omega(m)&\\
\IEEEeqnarraydblrulerow\\
\IEEEeqnarrayseprow[3pt]\\
&x < 0&&\Omega(m)=\sum\limits_{i=0}^{m}K^{-i}&\IEEEe
qnarraystrutsize{0pt}{0pt}\\
\IEEEeqnarrayseprow[3pt]\\
\IEEEeqnarrayrulerow\\
\IEEEeqnarrayseprow[3pt]\\
&x \ge 0&&\Omega(m)=\sqrt{m}\hfill&\IEEEeqnarraystru
tsize{0pt}{0pt}\\
\IEEEeqnarrayseprow[3pt]\\
\IEEEeqnarrayrulerow
\end{IEEEeqnarraybox}
\end{table}
```

The solution is to use `\IEEEeqnarrayseprow` to manually add in a fixed amount of extra space as needed. In this way, `\IEEEeqnarrayseprow` can do for lined tables what `\jot` does for multiline equations. Of course, using this method, the baselines of the rows will no longer be equally spaced.

The `\hfill` in the square root cell is a cheap, but effective, way of getting the equal signs to line up without the need of additional columns.

ACKNOWLEDGMENTS

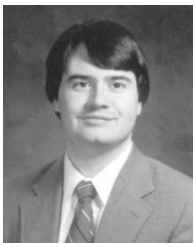
The author would like to thank Ken Rawson, Kevin Lisankie, Kimberly Sperka, Steve Wareham, Patrick Kellenberger, Laura Hyslop and Cathy Cardon of the IEEE for their help and support in making this work possible. The knowledge and prior work of TEX gurus such as Donald Arseneau, Fred Bartlett, David Carlisle, Tony Liu, Frank Mittelbach, Piet van Oostrum, Roland Winkler and Mark Wooding were instrumental in developing the complex IEEEeqnarray family of commands. The author is also grateful to Peter Wilson and Donald Arseneau for allowing the inclusion of their `\@ifmtarg` command.

Finally, this work might not have been possible had it not been for the efforts of the prior IEEEtran developers: Gerry Murray, Silvano Balemi, Jon Dixion, Peter Nüchter and Juergen von Hagen. Their work still lives on to some degree within IEEEtran.

REFERENCES

[1] (2015, Jul.) The IEEE website. [Online]. Available: http://www.ieee.org/
[2] M. Shell. (2015, Aug.) The IEEEtran.cls package. [Online]. Available: http://www.ctan.org/pkg/ieeetran
[3] ——. (2015, Jul.) IEEEtran homepage. [Online]. Available: http://www.michaelshell.org/tex/ieeetran/
[4] H. Kopka and P. W. Daly, *Guide to LATEX*, 4th ed. Harlow, England: Addison-Wesley, 2003.
[5] T. Oetiker, H. Partl, I. Hyna, and E. Schlegl. (2015, Jul.) The not so short introduction to LATEX 2$_\varepsilon$. [Online]. Available: http://www.ctan.org/pkg/lshort
[6] S. M. Moser. (2013, Aug.) How to Typeset Equations in LATEX. [Online]. Available: http://moser.cm.nctu.edu.tw/manuals.html#eqlatex
[7] R. Fairbairns. (2014, Jun.) The TEX FAQ. [Online]. Available: http://www.tex.ac.uk/
[8] M. Sharpe. (2015, Jul.) The newtx package. [Online]. Available: http://www.ctan.org/pkg/newtx
[9] (2015, Jul.) Mathtime professional fonts. Personal TEX, Inc. [Online]. Available: http://www.pctex.com/mtpro2.html
[10] D. Carlisle and F. Mittelbach. (2015, Apr.) The bm package. [Online]. Available: http://www.ctan.org/pkg/bm
[11] (2013, Jan.) The amsmath package. The American Mathematical Society. [Online]. Available: http://www.ctan.org/pkg/amsmath
[12] S. Pakin. (2009, Apr.) The IEEEconf.cls package. [Online]. Available: http://www.ctan.org/pkg/ieeeconf
[13] J. D. McCauley, J. Goldberg, and A. Sommerfeldt. (2011, Dec.) The endfloat package. [Online]. Available: http://www.ctan.org/pkg/endfloat
[14] H. Oberdiek. (2012, May) The ifpdf package. [Online]. Available: http://www.ctan.org/pkg/ifpdf
[15] A. Gefen, "Simulations of foot stability during gait characteristic of ankle dorsiflexor weakness in the elderly," *IEEE Trans. Neural Syst. Rehab. Eng.*, vol. 9, no. 4, pp. 333–337, Dec. 2001.
[16] D. Arseneau. (2015, Mar.) The cite package. [Online]. Available: http://www.ctan.org/pkg/cite
[17] M. D. Wooding. (1999, Mar.) The MDW tools package. [Online]. Available: http://www.ctan.org/pkg/mdwtools
[18] D. Arseneau. (2010, Feb.) The cases package. [Online]. Available: http://www.ctan.org/pkg/cases
[19] S. Tolušis and V. Statulevičius. (2013, Oct.) The stfloats package. [Online]. Available: http://www.ctan.org/pkg/stfloats
[20] D. Carlisle. (2015, Apr.) Packages in the 'graphics' bundle. grfguide.pdf. [Online]. Available: http://www.ctan.org/pkg/graphics
[21] K. Reckdahl. (2006, Jan.) Using imported graphics in LATEX 2$_\varepsilon$. [Online]. Available: http://www.ctan.org/pkg/epslatex
[22] C. Barratt, M. C. Grant, and D. Carlisle. (1998, May) The psfrag package. [Online]. Available: http://www.ctan.org/pkg/psfrag
[23] S. D. Cochran. (2005, Jul.) The subfigure package. [Online]. Available: http://www.ctan.org/pkg/subfigure
[24] S. D. Cochran, V. Karen-Pahlav, Z. Mehran, and V. Khalighi. (2005, Jul.) The subfig package. [Online]. Available: http://www.ctan.org/pkg/subfig
[25] A. Sommerfeldt. (2013, May) The subcaption package. [Online]. Available: http://www.ctan.org/pkg/subcaption
[26] P. Williams and R. Brito. (2009, Aug.) The algorithmic package. [Online]. Available: http://www.ctan.org/pkg/algorithms
[27] S. János. (2005, Apr.) The algorithmicx.sty package. [Online]. Available: http://www.ctan.org/pkg/algorithmicx
[28] F. Mittelbach and D. Carlisle. (2015, Apr.) The array package. [Online]. Available: http://www.ctan.org/pkg/array
[29] D. Arseneau. (2010, Mar.) The threeparttable package. [Online]. Available: http://www.ctan.org/pkg/threeparttable
[30] D. Carlisle. (1999, Apr.) The fix2col package. [Online]. Available: http://www.ctan.org/pkg/fix2col
[31] M. Høgholm. (2012, Dec.) The dblfloatfix package. [Online]. Available: http://www.ctan.org/pkg/dblfloatfix
[32] M. Shell. (2015, Aug.) The IEEEtran BIBTEX style. [Online]. Available: http://www.ctan.org/pkg/ieeetran
[33] P. W. Daly. (2013, May) The balance package. [Online]. Available: http://www.ctan.org/pkg/balance
[34] S. Tolušis and V. Statulevičius. (2015, Apr.) The flushend package. [Online]. Available: http://www.ctan.org/pkg/flushend
[35] M. Shell. (2007, Jan.) The testflow diagnostic suite. [Online]. Available: http://www.ctan.org/pkg/testflow

[36] T. Oetiker. (2015, Mar.) The acronym package. [Online]. Available: http://www.ctan.org/pkg/acronym
[37] D. Arseneau. (2013, Dec.) The url package. [Online]. Available: http://www.ctan.org/pkg/url
[38] M. Shell. (2015, Aug.) The IEEEtrantools package. [Online]. Available: http://www.ctan.org/pkg/ieeetrantools



Michael Shell (M'87) received the B.E.E., M.S.E.E. and Ph.D. degrees in electrical engineering all from the Georgia Institute of Technology, Atlanta, in 1991, 1993 and 2004 respectively. He has developed several all-optical packet-switched network subsystems and node demonstrations. His research interests include all-optical packet-switched networks, high speed opto-electronic interface design, discrete simulation and exact Markov models for buffered packet switches.

Dr. Shell is also the author of the most recent versions of the IEEEtran LaTeX class and BibTeX style packages and is the current maintainer of both.